TECHNISCHE UNIVERSITÄT DARMSTADT

Machine Learning for Materials Science

Introduction to local descriptors of atomic environments and how to connect them to atomic properties through machine-learning. Hands-on session: How to train simple models for atomic properties (NMR shifts)

Agenda for Today

9:00	Motivation: Why do we need machine-learning in atomic simulations?
9:15	Introduction to atomic descriptors for machine learning on the atomic level
10:00	Break
10:15	Hands-on Tutorial: Make your own machine-learning model for NMR Shifts
11:00	Break
11:15	Machine Learning Potential: How to generate a database?
11:30	The Atomic Cluster Expansion: How to fit a potential?
12:45	Short feedback round

TECHNISCHE UNIVERSITÄT DARMSTADT

Installation on Google Colab

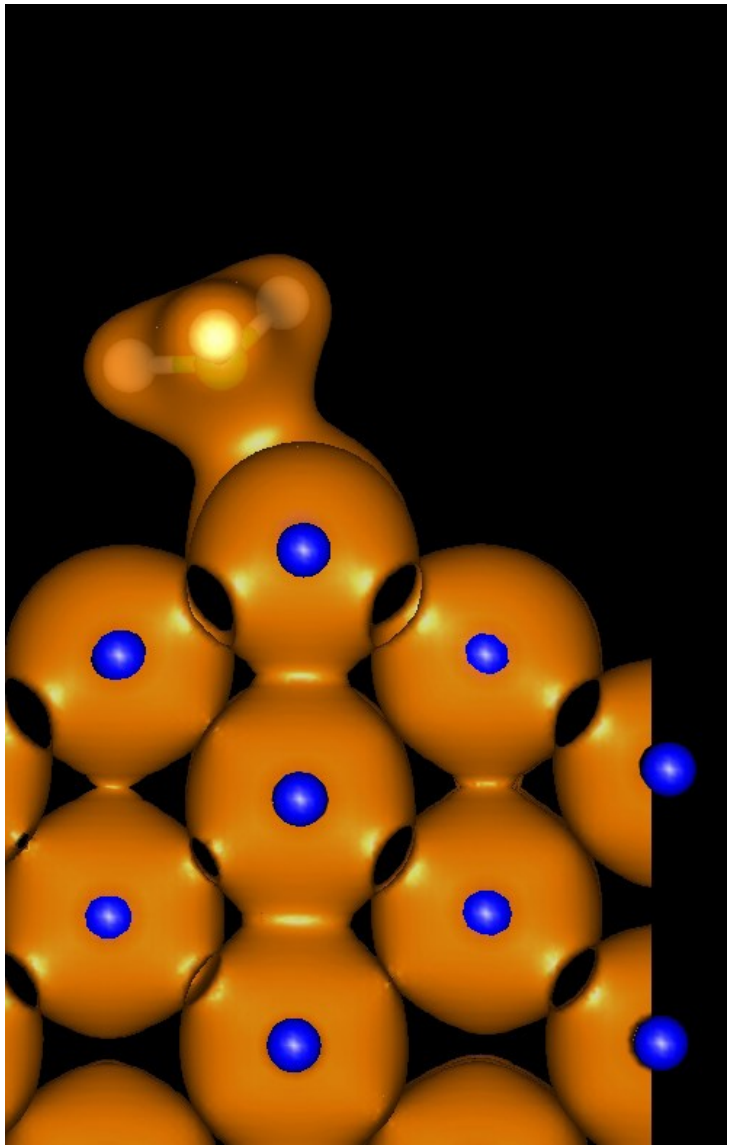
Motivation

How to describe solid state materials

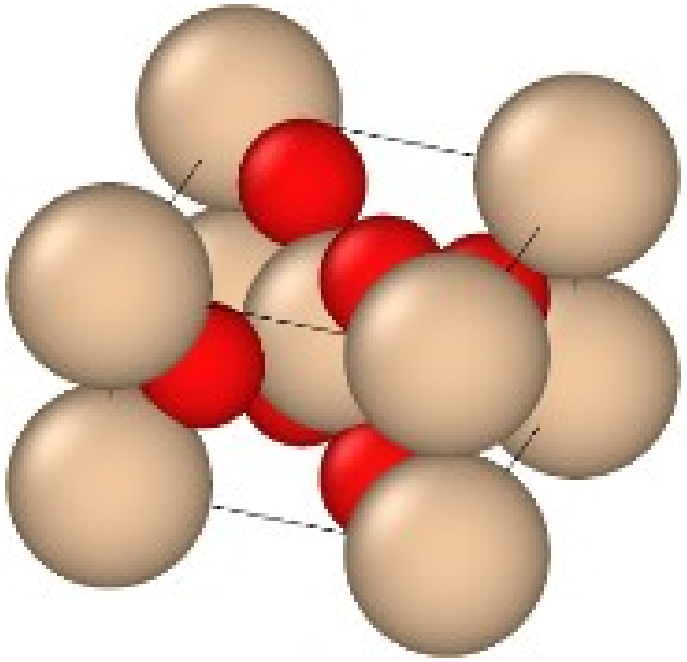
TECHNISCHE UNIVERSITÄT DARMSTADT

Let‘s start with the “standard model of matter”:

- Atoms are made by massive, point-like nuclei (protons+neutrons)
- Surrounded by tightly bound, rigid shells of core electrons
- Bound together by a glue of valence electrons – creating bonds



Atomic Properties of Interest



We are interested into various properties of atomic structures:

- Energy
- Forces (Derivative of the potential)
- Other things e.g., (NMR shifts, phonon dispersion,…)

Simulations

TECHNISCHE UNIVERSITÄT DARMSTADT

Two main approaches:

- Quantum Mechanics → Density Function Theory (DFT)
- Classical Mechanics → Interatomic Potential

Quantum mechanical approach

Quantum Mechanical Many-Body Problem

TECHNISCHE UNIVERSITÄT DARMSTADT

- If we want to describe a solid based on quantum mechanics we simply need to solve the Schrödinger equation for a system of ***electrons at positions x_i and nuclei at positions r_i:***

$$\hat{H}\Psi(\vec{x}_1,..,\vec{x}_{N^{el}},\vec{r}_1,..,\vec{r}_{N^{Nuc}},t) = i\hbar\frac{\partial\Psi}{\partial t}$$

- The Hamiltonian operator is simply given by the kinetic energy operators (*T)* for the electrons and nuclei and the potential energy terms *(U)* representing the electron-electron U^{ee}, nucleus-nucleus U^{ZZ} and electron-nucleus U^{Ze} Coulomb interaction:

$$\hat{H} = \underbrace{\sum_{i=1}^{N^{el}} \frac{\hbar^2}{2m^{el}} \vec{\nabla}_i^{\,2}}_{\hat{T}^e} + \underbrace{\sum_{\alpha=1}^{N^{Nuc}} \frac{\hbar^2}{2m^{\alpha}} \vec{\nabla}_{\alpha}^{\,2}}_{\hat{T}^Z} + \underbrace{\sum_{\alpha>\beta}^{N} \frac{e^2 Z^{\alpha} Z^{\beta}}{|\vec{r}^{\alpha} - \vec{r}^{\beta}|}}_{U^{ZZ}} + \underbrace{\sum_{i=1}^{N^{el}} \sum_{\beta}^{N} \frac{-e^2 Z^{\beta}}{|\vec{x}_i - \vec{r}^{\beta}|}}_{U^{Ze}} + \underbrace{\sum_{i>j}^{N^{el}} \frac{e^2}{|\vec{x}_i - \vec{x}_j|}}_{U^{ee}}$$

Electrons and Nuclei

TECHNISCHE UNIVERSITÄT DARMSTADT

- Within the Born-Oppenheimer approximation we arrive at a time-independent Schrödinger equation for the electrons:

$$(\hat{T}^e + U^{ee} + U^{Ze})\Psi^{el} = \varepsilon\Psi^{el}$$

- For the ground state we get:

$$(\hat{T}^e + U^{ee} + U^{Ze})\Psi_0{}^{el} = \varepsilon_o(\vec{r}_1, .., \vec{r}_{N^{Nuc}})\Psi_o{}^{el}$$

- Notice, the ground state energy of the electrons depends only on the positions of the nuclei

Electrons and Nuclei

TECHNISCHE UNIVERSITÄT DARMSTADT

- For the nuclei the time-dependent Schrödinger equation reads within the Born-Oppenheimer approximation as:

$$(\hat{T}^Z + U^{ZZ} + \varepsilon_o[\vec{r}_1, .., \vec{r}_{N^{Nuc}}])\Psi^{Nuc} = i\hbar\frac{\partial}{\partial t}\Psi^{Nuc}$$

- This equation is essentially describing the motion of nuclei in the presence of a potential field U given by the Coulomb interaction of the nuclei and ε_0, a function of the nuclear positions:

$$U(\vec{r}_1, .., \vec{r}_{N^{Nuc}}) = U^{ZZ} + \varepsilon_o[\vec{r}_1, .., \vec{r}_{N^{Nuc}}]$$

Many body problem to DFT

- Introduce a functional form of the wavefunction

$$\rho(\mathbf{r}) = N_e \sum_{s_1} \cdots \sum_{s_{N_e}} \int d\mathbf{x}_2 \cdots d\mathbf{x}_{N_e} \left|\Psi(\mathbf{r}, s_1, \mathbf{x}_2, s_2, \ldots, \mathbf{x}_{N_e}, s_{N_e})\right|^2,$$

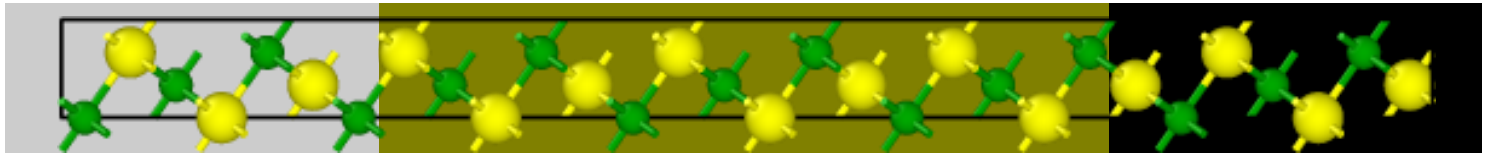
so as to, $E[\rho_0] = \min E[\rho]$ Two theorems proven by Hohenberg-Kohn

- The Schrodinger equation can be written: $E[\rho] = T[\rho] + V_{ext}[\rho] + V_{ee}[\rho]$

$$V_{ext}[\rho] = \int \hat{V}_{ext}\, \rho(\mathbf{r}) d\mathbf{r}$$

- Within the framework of Kohn-Sham DFT, the intractable many-body problem of interacting electrons in a static external potential is reduced to a tractable problem of non-interacting electrons moving in an effective potential.

Kohn-Sham DFT



Solve KS (non-interacting) Schrödinger equation for a functional → Get solution – **Total Energy** and Forces

Optimize density ρ until convergence

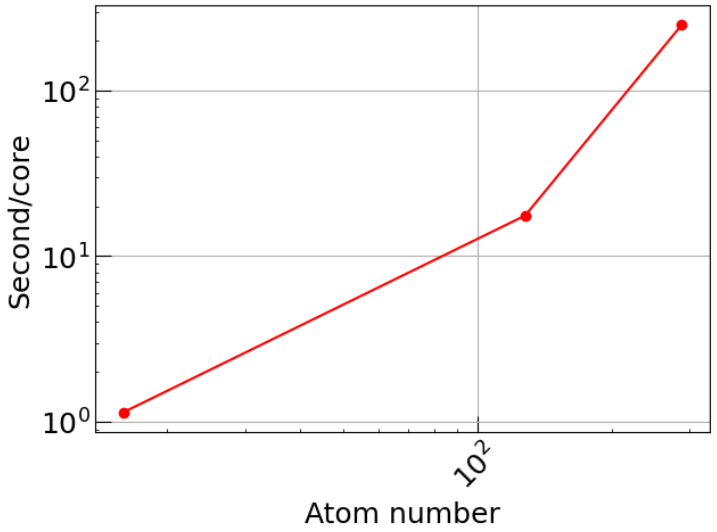
Quantum Mechanical Calculations – Density functional Theory (DFT)

Schrödinger equation → Energy and Forces

- Full electronic calculations
- Charge transfer
- Van der Waals interactions
- ⋮ ⋮ ⋮ ⋮ ⋮ ⋮ ⋮

b-initio VASP ackage ienna imulation

System size $10^1 - 10^3$



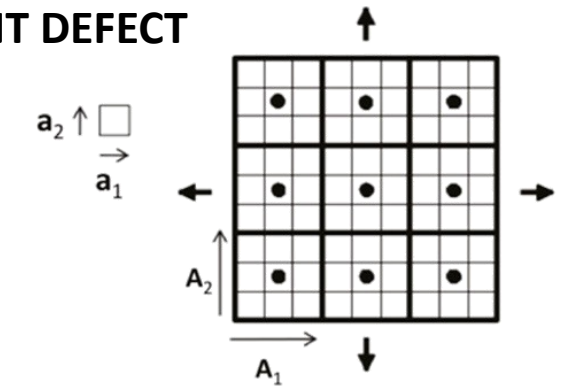
DFT calculations varying structure atomic size

Supercells

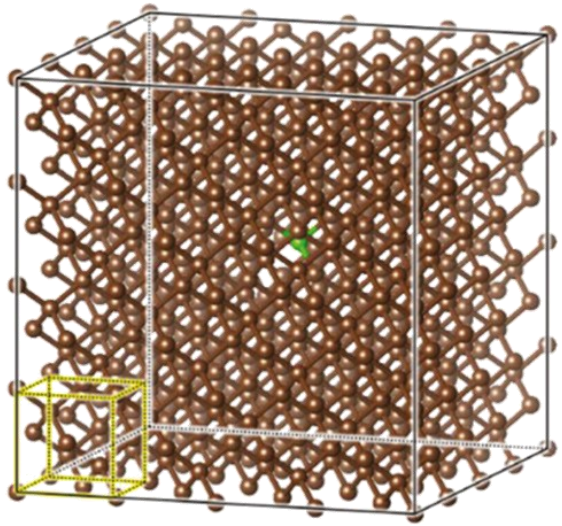
TECHNISCHE UNIVERSITÄT DARMSTADT

Supercells are periodic cells replicated in all direction, which help to use Bloch waves as wave functions:

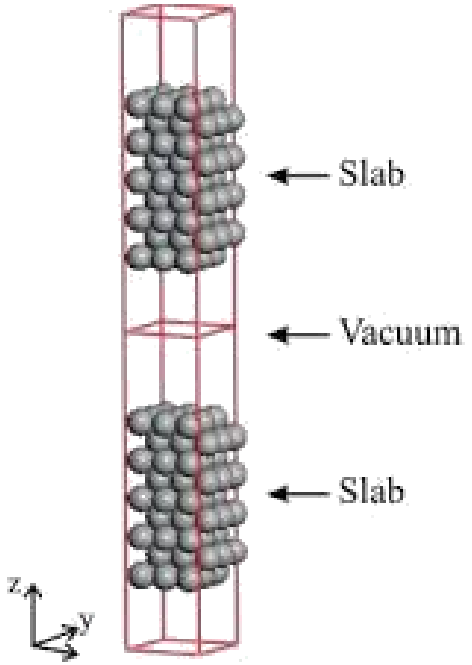
POINT DEFECT



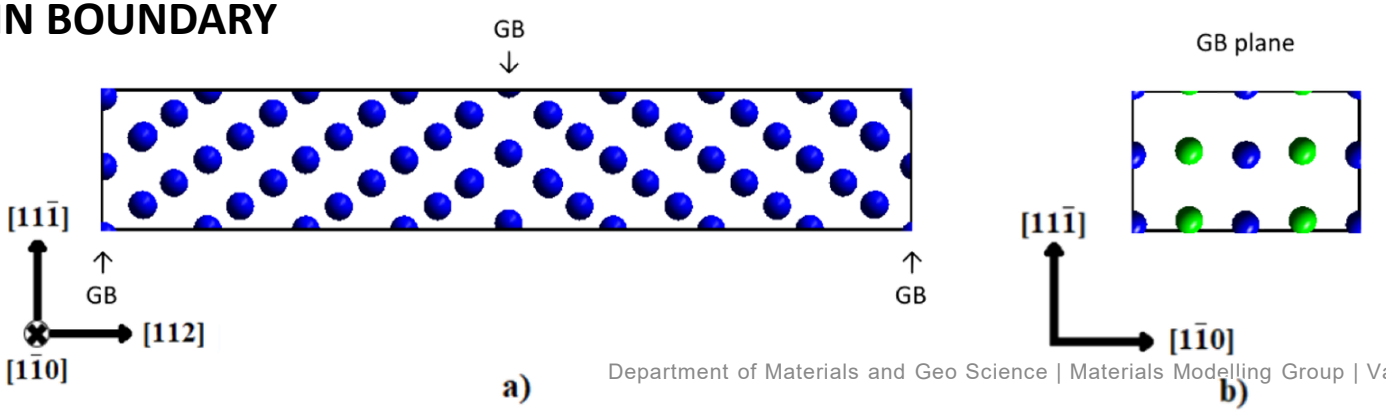
$A_i = T_{ij} a_j$; $i, j = 1, 2, 3$



SURFACE



GRAIN BOUNDARY



Classical approaches to get potential energy

Classical mechanics

TECHNISCHE UNIVERSITÄT DARMSTADT

- Equation of motion of particle i, with mass m, interacting with $N-1$ particles

$$\frac{d\mathbf{p}_i}{dt} = \mathbf{F}_E + \sum_{i \neq j} \mathbf{F}_{ij} \qquad F_{ij} = \frac{\partial U_{ij}}{\partial x_i}$$

- U_{ij} is the potential energy that fully characterizes the equation of motion

Interatomic potential

- The total potential energy can always be formally expressed as an expansion on structural motifs of the following form:

$$V = \phi_o + \sum_{i=1}^{N} \phi_i(\vec{r}_i) + \frac{1}{2}\sum_{i,j} \phi_{ij}(\vec{r}_i, \vec{r}_j) + \frac{1}{3!}\sum_{i,j,k} \phi_{ijk}(\vec{r}_i, \vec{r}_j, \vec{r}_k) + \ldots$$

- The first term is the reference energy corresponding to the sum of the eigenenergies of N fully isolated atoms (or ions):

$$\phi_o = \sum_{i=1}^{N} E_{free}(Z^i)$$

- The second term is describing the influence of an external (magnetic, gravimetric, electric …) field:

$$V^{ext} = \sum_{i=1}^{N} \phi_i(\vec{r}_i)$$

Interatomic potential

- We can split the potential into an external and internal part:

$$V = U + V^{ext}$$

$$U = \frac{1}{2}\sum_{i,j}\phi_{ij}(\vec{r}_i,\vec{r}_j) + \frac{1}{3!}\sum_{i,j,k}\phi_{ijk}(\vec{r}_i,\vec{r}_j,\vec{r}_k) + \ldots$$

- U is called the **INTERATOMIC POTENTIAL**
- Note, that the different potential terms $\Phi_{ij,}$ $\Phi_{ijk,}$ ..can depend on the atom type of atoms *i,j,k !*

Categorization of Interatomic Potentials

TECHNISCHE UNIVERSITÄT DARMSTADT

- A.E. Carlsson (Sol. Stat. Phys. 43 (1990) 1-91) proposed the following categorization of interatomic potentials:

Pair Potentials:

$$U = \sum_{i>j} \phi_{ij}(\vec{r}_i, \vec{r}_j)$$

Cluster Potentials:

$$U = \sum_{i>j} \phi_{ij}(\vec{r}_i, \vec{r}_j) + \sum_{i>j>k} \phi_{ijk}(\vec{r}_i, \vec{r}_j, \vec{r}_k)$$

Pair Functional:

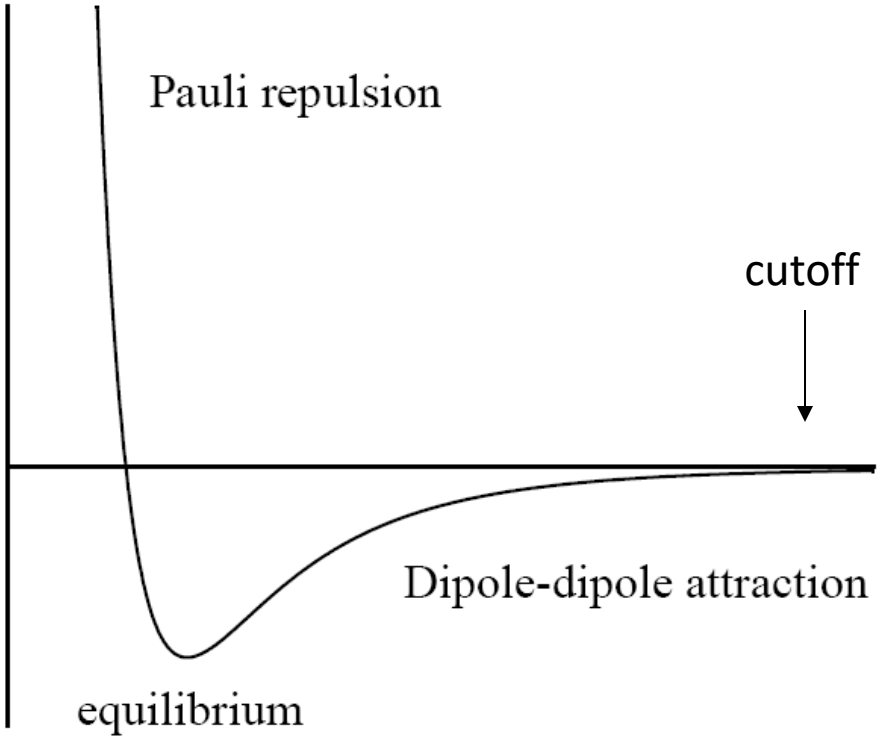
$$U = \sum_{i>j} \phi_{ij}(\vec{r}_i, \vec{r}_j) + \sum_{i} F_i \left(\sum_{j} g_{ij}(\vec{r}_i, \vec{r}_j) \right)$$

Cluster Functionals:

$$U = \sum_{i>j} \phi_{ij}(\vec{r}_i, \vec{r}_j) + \sum_{i} F_i \left(\sum_{j} g_{ij}(\vec{r}_i, \vec{r}_j), \sum_{j>k} g_{ijk}(\vec{r}_i, \vec{r}_j, \vec{r}_k), \dots \right)$$

Pair Potentials: Lennard-Jones

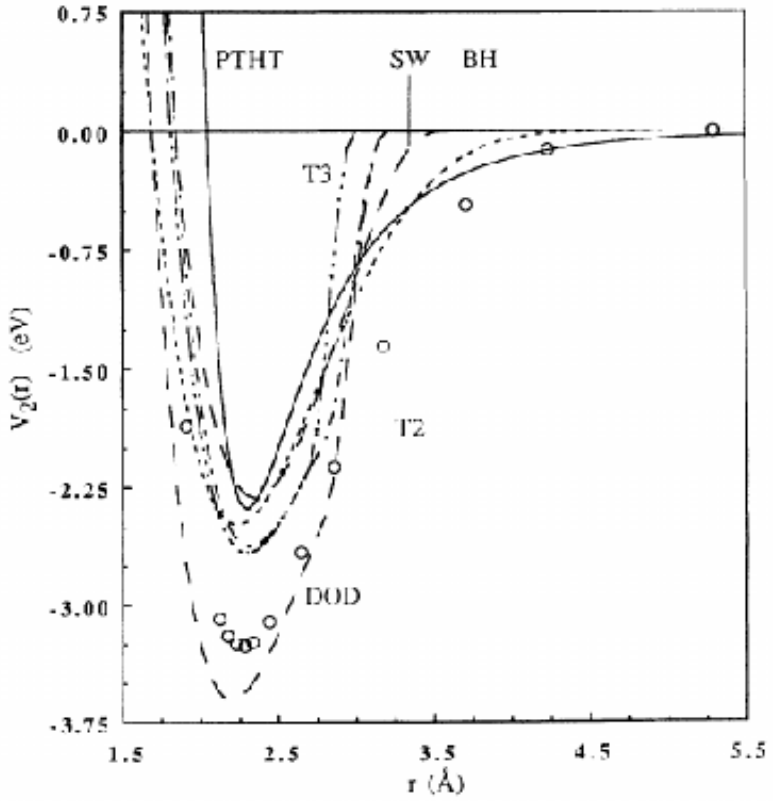
$$\phi_{ij}(r_{ij}) = 4\varepsilon\left(\left[\frac{\sigma}{r_{ij}}\right]^{12} - \left[\frac{\sigma}{r_{ij}}\right]^{6}\right)$$



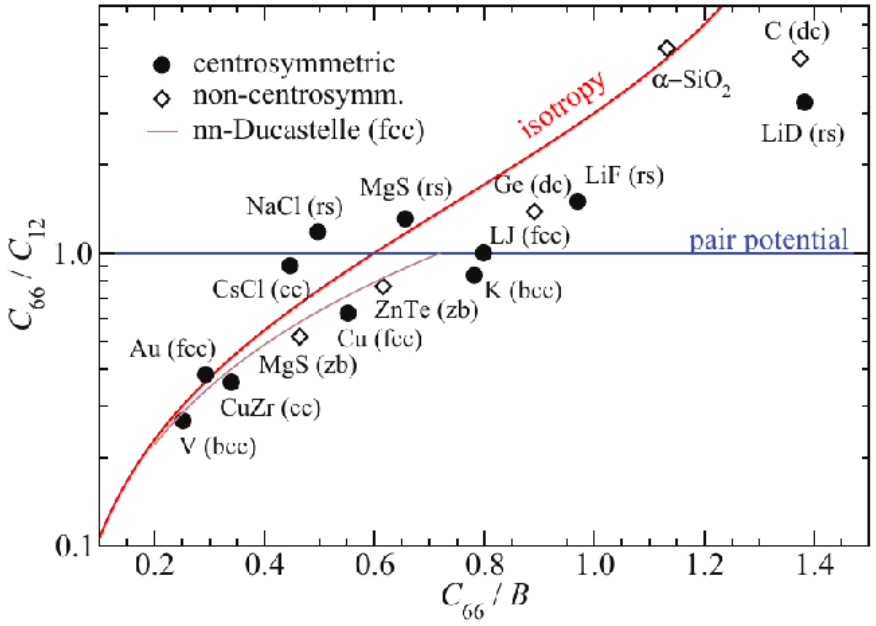
- Potential functions like L-J have an infinite range of interaction. In practice a cutoff radius R_c established and interactions between atoms separated by more than R_c are ignored for two reasons:
- The number of pair interactions grows as N^2. Example: Consider system of 3000 atoms. There are $N^2/2$ = 4.5 million pairs of atoms. Using a cutoff of 8-10 A we can reduce a number of interacting neighbors for each atom to ~50 and and we will have to evaluate force only ~50N = 150 thousand times.
- The size of the system that can be simulated is finite. Periodic boundary conditions are often used and we do not want an atom to interact with itself.

Classical Interatomic Potentials (CIPs)

TECHNISCHE UNIVERSITÄT DARMSTADT



Comparison of the two-body potential functions $V_2(r)$ for silicon. The open circles correspond to ab initio calculations.



Ratio of elastic tensor elements C_{66} and C_{12} as a function of C_{66} in units of the bulk modulus B for a various systems.

Balamane et al., Phys. Rev. B 2250, 46 (1992)
Muser et al., Advances in Physics, 2093129 (2023)

Classical interatomic potentials

TECHNISCHE UNIVERSITÄT DARMSTADT

Approximate potential landscape → Energy and Forces

Diffusion effects

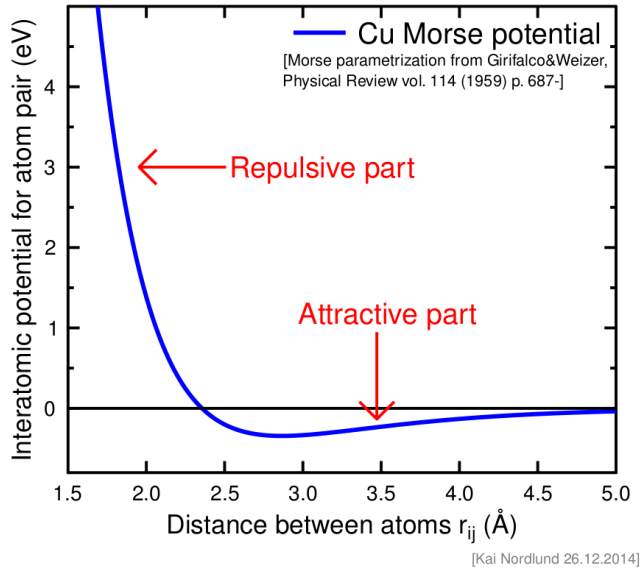
Structural exploration

Melting temperatures

⋮ ⋮ ⋮ ⋮ ⋮ ⋮ ⋮

LAMMPS
Large-scale Atomic/Molecular Massively Parallel Simulator

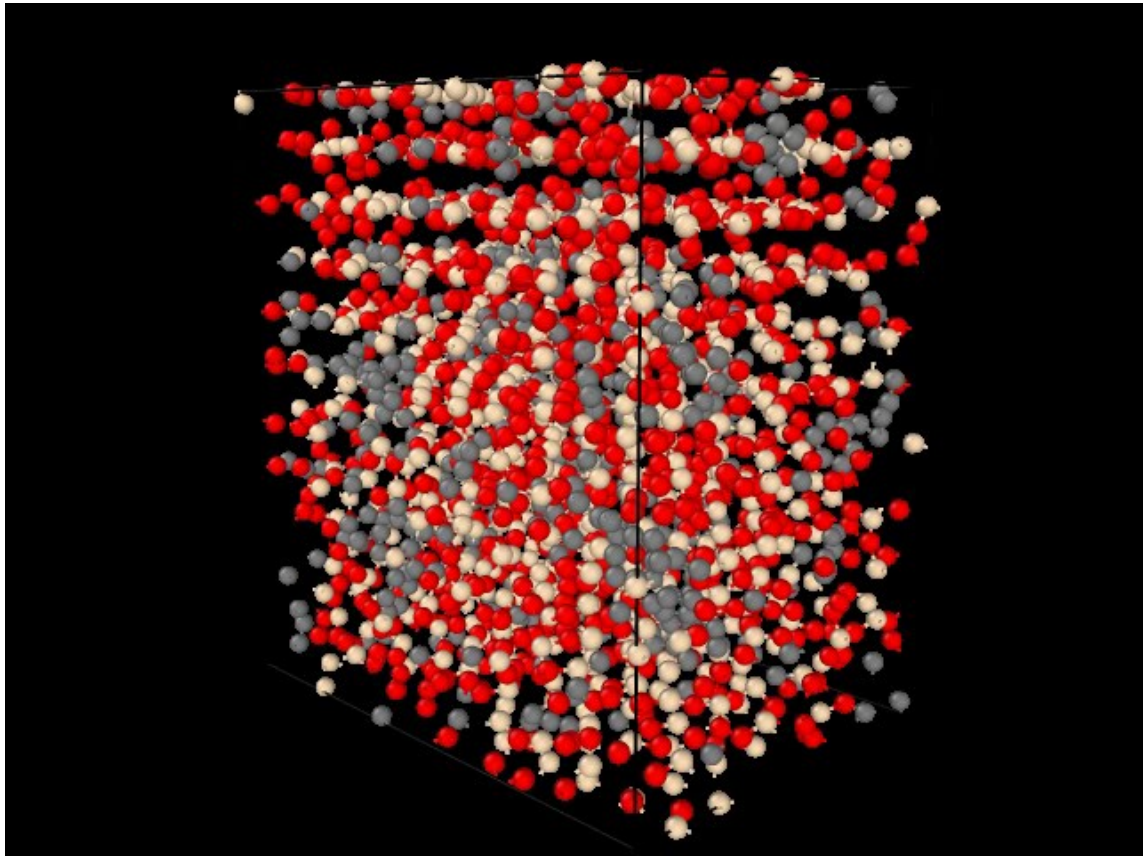
System size
$10^1 - 10^6$



- Provide "simple" formula to calculate the energy
- Forces can be accessed by the derivative of this formula

Simulations with Si-O-C

TECHNISCHE UNIVERSITÄT DARMSTADT



Anneal @ 1800 K

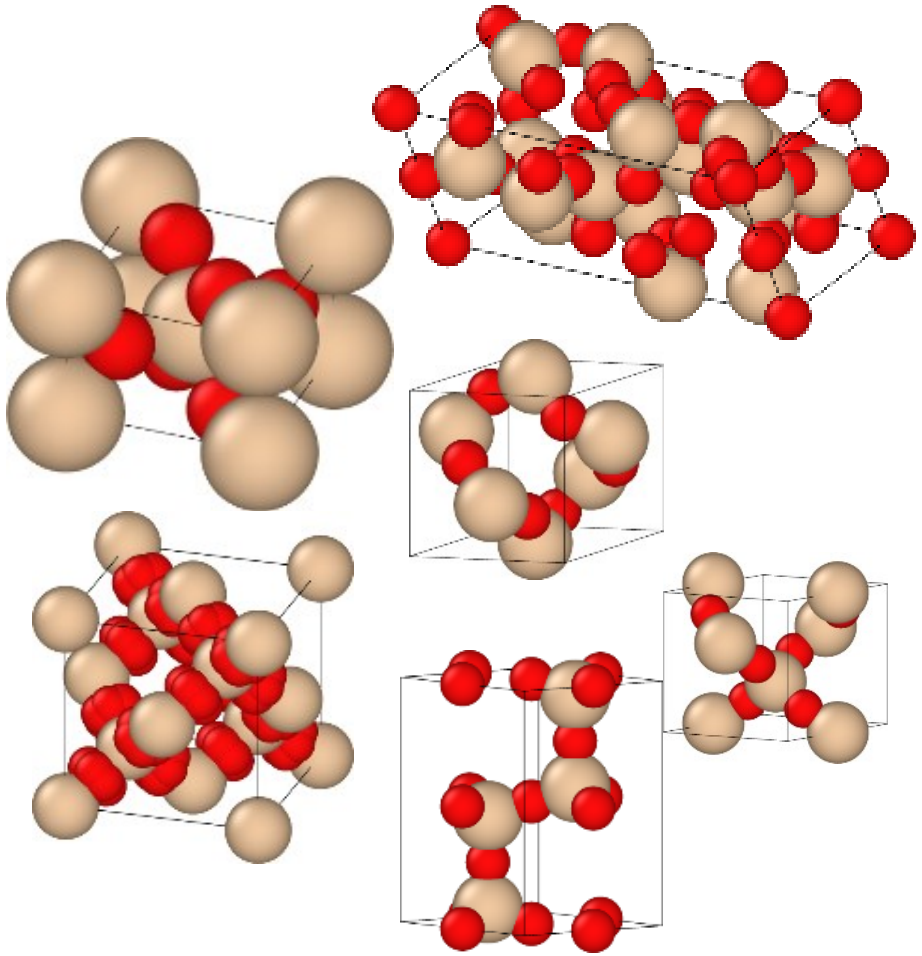
Problems

1. What are we doing if we need DFT accuracy on forces and energies but on **systems larger than 1000 atoms**?

2. What if we need **much faster evaluations** times than DFT?

3. What if we are interested in **other properties** than forces and energy for large system sizes?

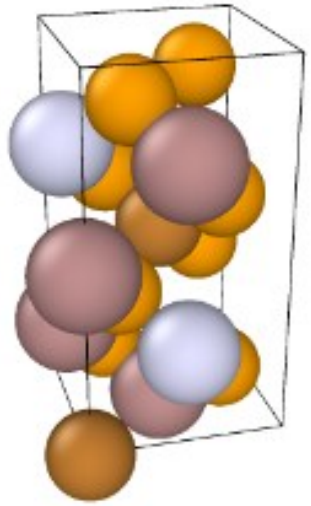
So why not using machine learning?



1. Generate a database with many structures labelled with DFT energies and forces (or other properties)
2. Put the database into machine-learning algorithm
3. Use the machine-learning as it is much faster than DFT

Atomic Descriptors

Machine learning approach



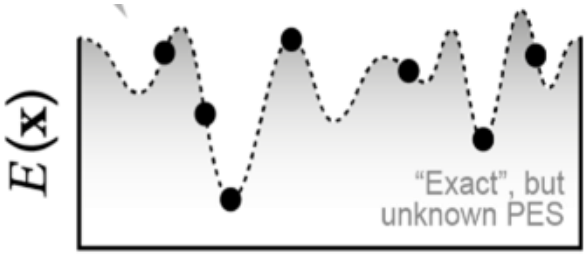
Atomic positions

Descriptors

"Fit"

Machine Learning Potential

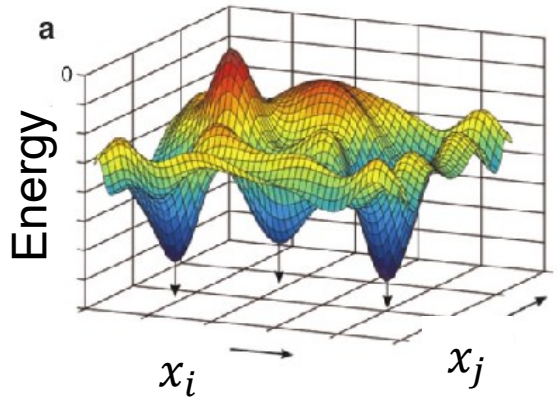
Energy and forces



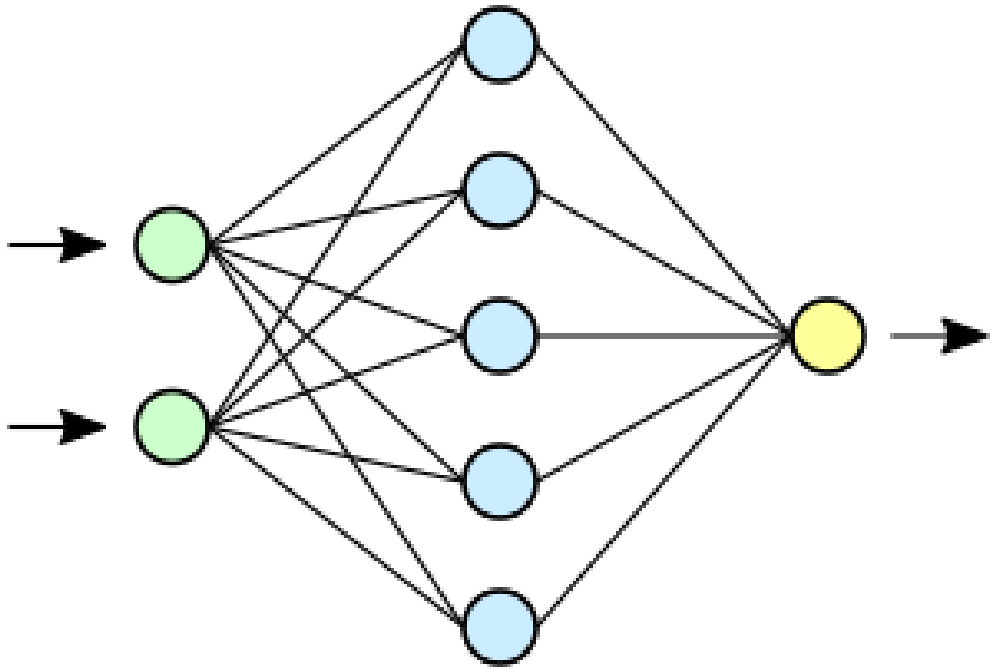
X (3N dimensional)

R. Drautz, Phys. Rev. B 99, 014104 (2019)

Energy landscape



Atomic Descriptors



Von Dake, Mysid - Vectorized by Mysid in CorelDraw on an image by Dake., CC BY 1.0, https://commons.wikimedia.org/w/index.php?curid=1412126

We cannot put the atomic coordinates into the ML algorithm:

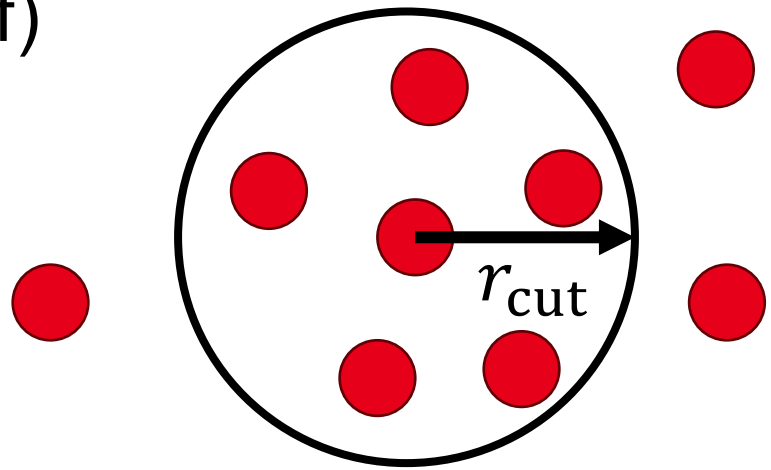
- Structures might have **different number of atoms**, however, we need always the same input
- The input needs to be **rotationally**, **translationally** and **permutationally invariant**

Some basic assumptions

- Decompose the total energy of the system to atomic energies

$$E = \sum_i E_i$$

- Each atomic energy does only depend on the atoms in the direct environment (< cutoff)



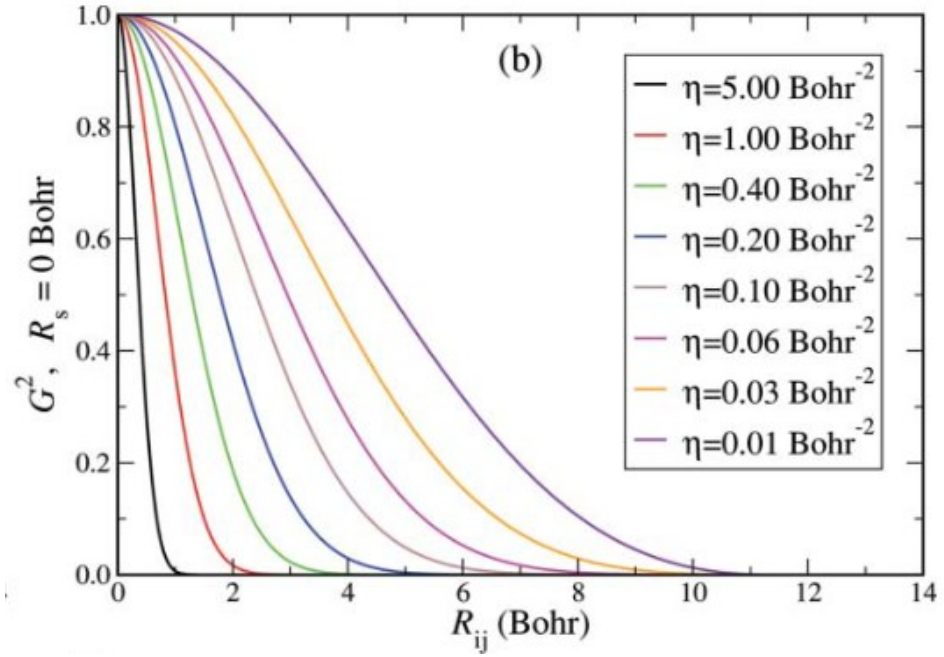
Behler and Parrinello, PRL 98, 146401 (2007)

Atom Centered Symmetry Functions (ACSF)

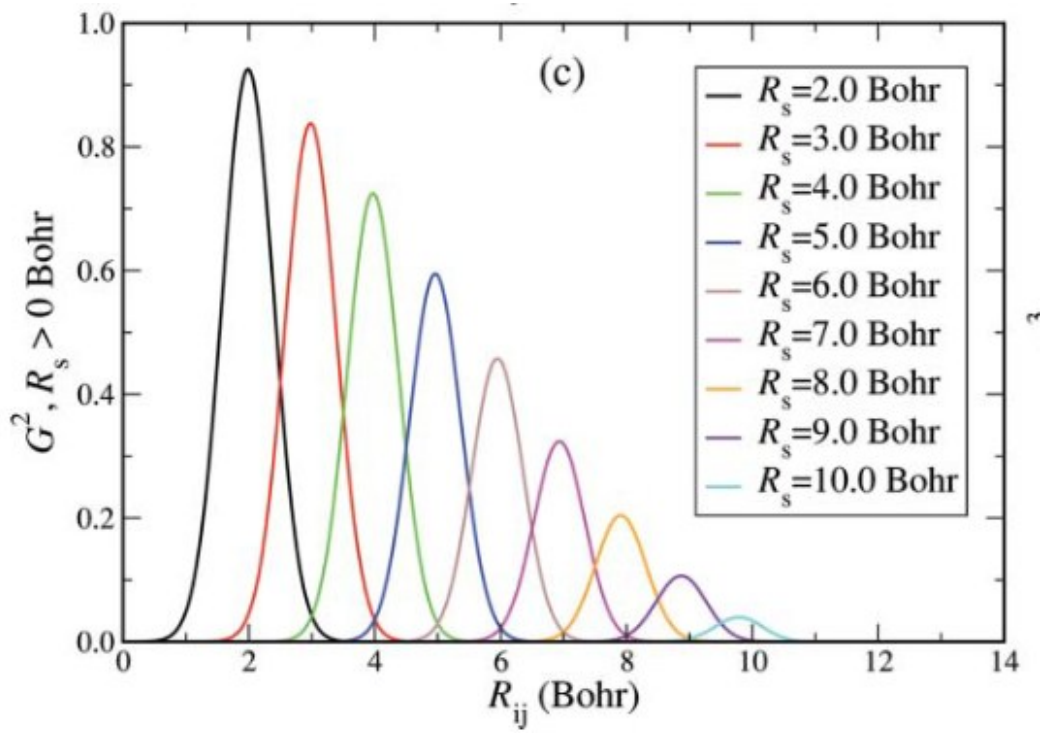
TECHNISCHE UNIVERSITÄT DARMSTADT

Cutoff function

Radial functions (2-body): $$G_i^2 = \sum_{j=1}^{N_{\text{atom}}} e^{-\eta\left(R_{ij}-R_s\right)^2} \cdot f_c\left(R_{ij}\right).$$



Behler and Parrinello, PRL 98, 146401 (2007)



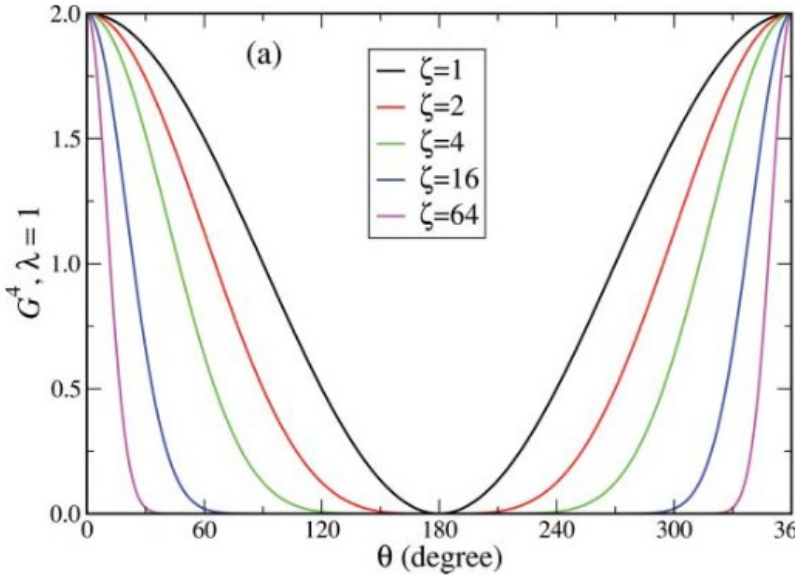
Behler, J. Chem. Phys. 134, 074106 (2011)

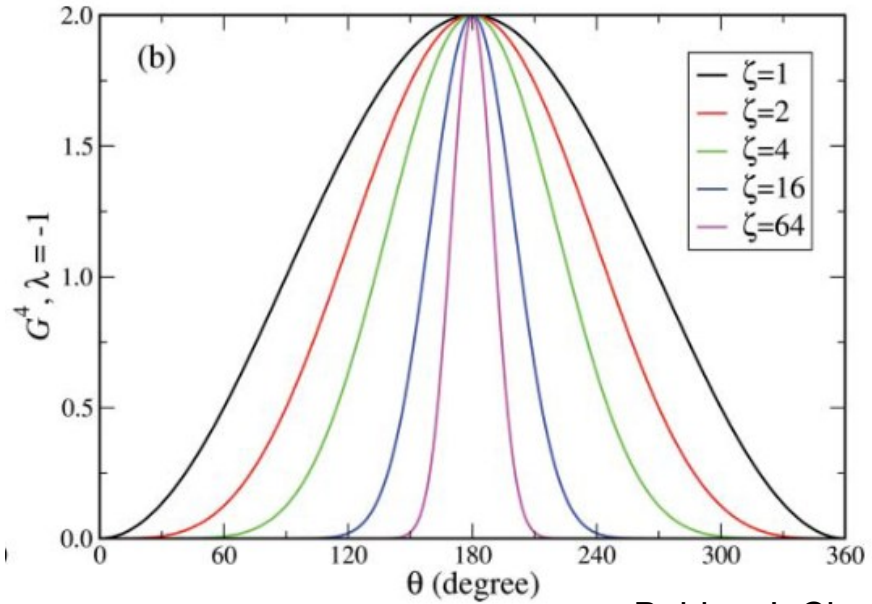
Atom Centered Symmetry Functions (ACSF)

TECHNISCHE UNIVERSITÄT DARMSTADT

Angular functions (3-body):

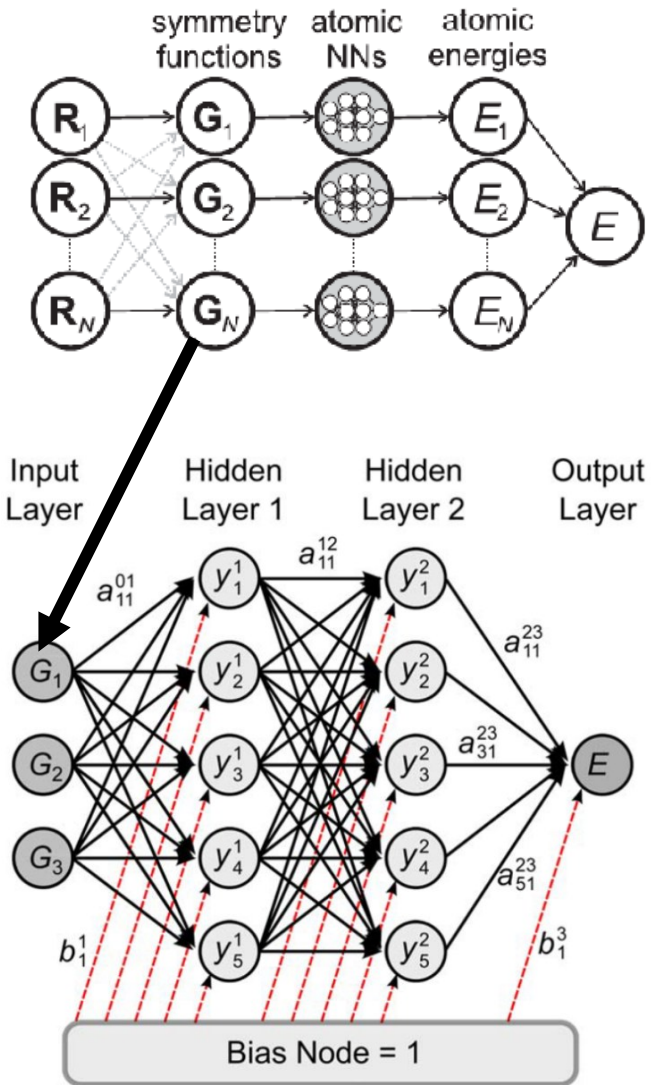
$$G_i^4 = 2^{1-\zeta} \sum_{j \neq i} \sum_{k \neq i,j} \left[\left(1 + \lambda \cdot \cos\theta_{ijk}\right)^{\zeta} \cdot e^{-\eta\left(R_{ij}^2 + R_{ik}^2\right)} \cdot f_c\left(R_{ij}\right) \cdot f_c\left(R_{ik}\right) \right].$$





Behler, J. Chem. Phys. 134, 074106 (2011)

Behler-Parrinello NNPs



- Use atom centered symmetry functions (ACSF) as input
- The input is used for a neural network with several layers
- The final output is the energy of the atom

Behler, J. Chem. Phys. 134, 074106 (2011)

Code example

Atom Centered Symmetry Functions

https://wiki.fysik.dtu.dk/ase/ase/atoms.html
https://wiki.fysik.dtu.dk/ase/ase/io/io.html

https://singroup.github.io/dscribe/latest/tutorials/descriptors/acsf.html

```
[8] from ase.io import read
```

```
[ ] atoms = read("data/training_SiO2.xyz")
```

```
[ ] from dscribe.descriptors import ACSF

    acsf = ACSF(
        species=["Si", "O"],
        r_cut=5.0,
        g2_params=[[1, 1], [1, 2], [1, 3]],
        g4_params=[[1, 1, 1], [1, 2, 1], [1, 1, -1], [1, 2, -1]],
        periodic=True
    )
```

```
[ ] acsf_functions = acsf.create(atoms)
```

```
[ ] acsf_functions[0]
```

```
    array([4.11313595e+00, 1.73824810e-01, 1.85101481e+00, 3.31550027e+00,
           2.50763238e+00, 1.05395555e+00, 1.36228348e+00, 7.13467543e-01,
           3.42298139e-10, 2.53060271e-10, 1.20876165e-10, 3.16382971e-11,
           1.14442382e-05, 1.03182328e-05, 1.24938850e-06, 1.23383121e-07,
           2.81666490e-08, 3.36896138e-09, 2.07401469e-07, 1.82603781e-07])
```

Smooth Overlap of Atomic Positions (SOAP)

TECHNISCHE UNIVERSITÄT DARMSTADT

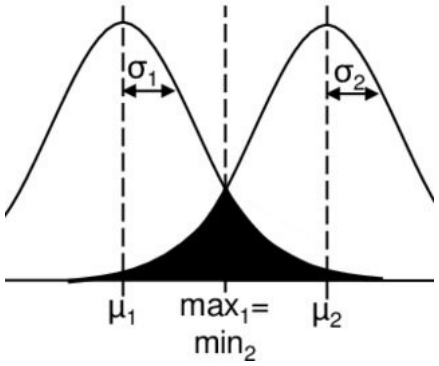
Similarity:

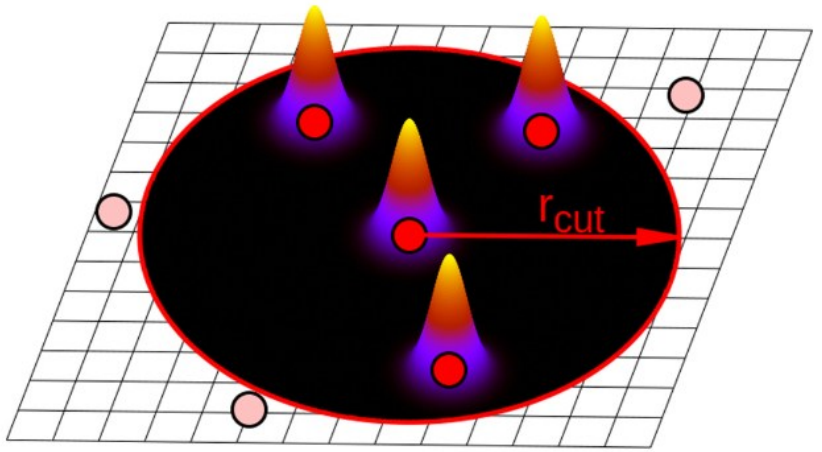
$$S(\rho, \rho') = \int \rho(\mathbf{r})\rho'(\mathbf{r})\mathrm{d}\mathbf{r}.$$

Similarity of the two atomic environments

Atomic density 1

Atomic density 2





Atomic density as a summation of Gaussians:

$$\rho(\mathbf{r}) = \sum_j \exp\left(-\frac{|\mathbf{r} - \mathbf{r}_j|^2}{2\sigma^2}\right)$$

Bartók et al., PRB 87, 184115 (2013)
Caro, PRB 100, 024112 (2019)

Smooth Overlap of Atomic Positions (SOAP)

TECHNISCHE UNIVERSITÄT DARMSTADT

Similarity is not rotationally invariant, yet.

→ Integrate over all possible rotations of the atomic density

$$k(\rho,\rho') = \int |S(\rho,\hat{R}\rho')|^n d\hat{R}$$

By normalisation we receive the SOAP kernel:

$$K(\rho,\rho') = \left(\frac{k(\rho,\rho')}{\sqrt{k(\rho,\rho)k(\rho',\rho')}}\right)^{\zeta},$$

Bartók et al., PRB 87, 184115 (2013)

SOAP Vector

The Gaussians can be expressed by spherical harmonics, making the calculation efficient

$$\exp\left(-\left|\mathbf{r}-\mathbf{r}_j\right|^2\right) \rightarrow C_{lm}^{n}\, J_n\, Y_{lm}$$

→The SOAP kernel can be expressed by a simple dot product:

$$K\left(\rho_i, \rho_j\right) = \vec{s}(\rho_i) \cdot \vec{s}(\rho_j)$$

SOAP vector → Descriptor for a single atom
Can be used for other ML techniques as input

Bartók et al., PRB 87, 184115 (2013)

Gaussian Approximation Potentials

TECHNISCHE UNIVERSITÄT DARMSTADT

- Gaussian approximation potentials use the SOAP Kernel within Gaussian Process Regression

- This is a kernel regression method

- The energy of an atom *i* is given by:

Number of structures in the database

Fitting coefficients extracted from training

$$E_i = \sum_{k=1}^{N} \alpha_k K\left(\rho_i, \rho_j\right)$$

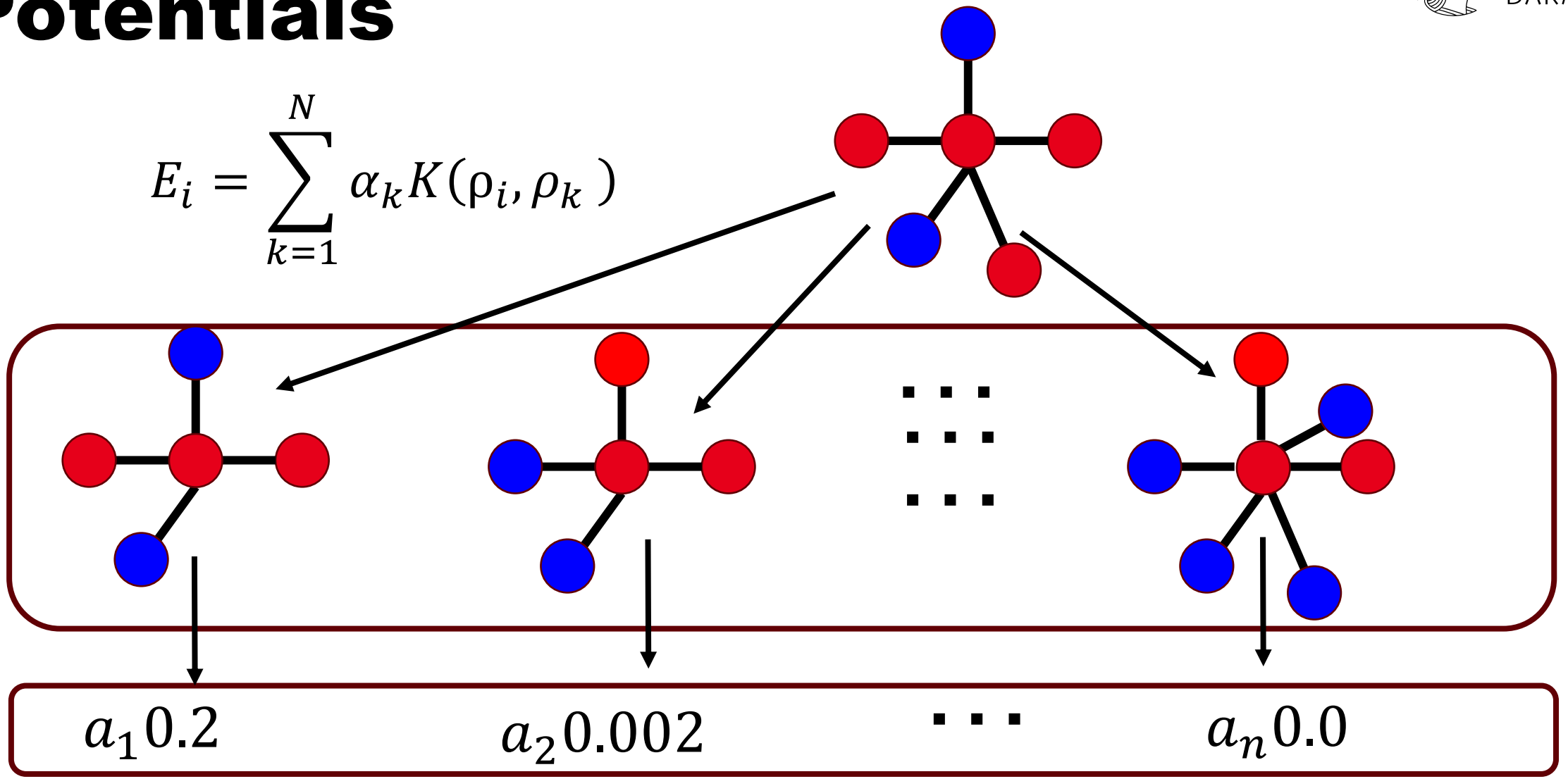
Compare your current structure with all structures in the database to get the energy

Bartók et al., PRB 87, 184115 (2013)

Gaussian Approximation Potentials

TECHNISCHE UNIVERSITÄT DARMSTADT

$$E_i = \sum_{k=1}^{N} \alpha_k K(\rho_i, \rho_k)$$



TECHNISCHE UNIVERSITÄT DARMSTADT

Hands on example

Code example

```
soap = SOAP(species=["Si","O"],periodic=False,r_cut=5,n_max=4,l_max=2)

soap_functions = soap.create(atoms)
soap_functions[0]
```

```
array([ 3.38465178e-02, -4.78858746e-01, -1.84983900e-01, -2.37171469e+00,
        6.77486826e+00,  2.61714246e+00,  3.35548943e+01,  1.01100632e+00,
        1.29623094e+01,  1.66192299e+02,  2.09835447e-01, -3.95369232e-01,
        1.18501900e+00, -2.32822471e+00,  7.46200841e-01, -2.23286742e+00,
        4.34940657e+00,  6.69320973e+00, -1.31442734e+01,  2.69550164e+01,
        6.00108286e-02, -1.29627067e-01,  2.99671220e-01, -4.99419948e-01,
        2.80764201e-01, -6.46735796e-01,  1.03629105e+00,  1.49831239e+00,
       -2.52195724e+00,  6.53630066e+00,  2.69256253e-02, -9.22468175e-02,
        5.21302118e-02, -1.09869247e+00, -3.80942326e-01,  1.30510310e+00,
       -7.37535484e-01,  1.55442431e+01, -1.47158630e-01,  5.04163418e-01,
       -2.84911138e-01,  6.00476599e+00, -1.88674952e+00,  6.46397759e+00,
       -3.65290131e+00,  7.69882768e+01,  1.05723776e-01, -2.03352304e-01,
        6.70453639e-01, -5.01207432e-01, -2.01265253e-01,  3.85779324e-01,
       -1.29111769e+00,  8.90430132e-01,  6.01426182e-01, -1.15887535e+00,
        3.79170440e+00, -2.94394429e+00, -1.10263794e+00,  2.15647369e+00,
       -6.59829926e+00,  6.93288416e+00,  2.95285237e-02, -6.37144715e-02,
        1.60048381e-01, -1.40528218e-01, -6.44962271e-02,  1.39036312e-01,
       -3.53192202e-01,  2.82975245e-01,  1.53060091e-01, -3.30540407e-01,
        8.23081369e-01, -7.67142154e-01, -1.89260198e-01,  4.15037860e-01,
       -8.35277517e-01,  2.17682477e+00,  2.14199081e-02, -7.33843069e-02,
        4.14706931e-02, -8.74033248e-01,  2.51413614e-01, -1.42078017e-01,
        2.99442574e+00,  8.02906519e-02, -1.69219982e+00,  3.56646777e+01,
        7.54395095e-02, -1.52388527e-01,  4.00694096e-01, -6.78159836e-01,
        3.13360139e-01, -7.49456712e-01,  1.62237982e+00,  2.77777786e+00,
       -8.66087897e-01,  1.76230270e+01,  4.77540024e-02, -1.03155338e-01,
        2.55719881e-01, -2.47484957e-01,  2.23032300e-01, -5.47982906e-01,
```

https://singroup.github.io/dscribe/latest/tutorials/descriptors/soap.html

Task

1.Take the **NMR database** (“nmr_shift.xyz”)

2. Load the database and calculate **SOAP vectors** for all **O atoms**, and get the corresponding O NMR shift for each O atoms (atoms.arrays[“nmr_shift”]). Split a train and test.

3. Create a **simple linear** ML Model, which takes the **SOAP vector as input** and the **NMR shifts as training property**

4. Calculate train and test errors, and calculate the **NMR spectra** using your ML model for the structure in (“large_sio2.xyz”)

5. (Optional) Play around with the SOAP vectors (increase l and n) or use the ACSF as descriptor.

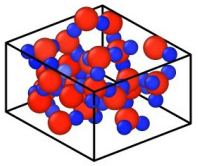
Helpful links:

https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html

https://scikit-learn.org/stable/modules/generated/sklearn.metrics.mean_squared_error.html

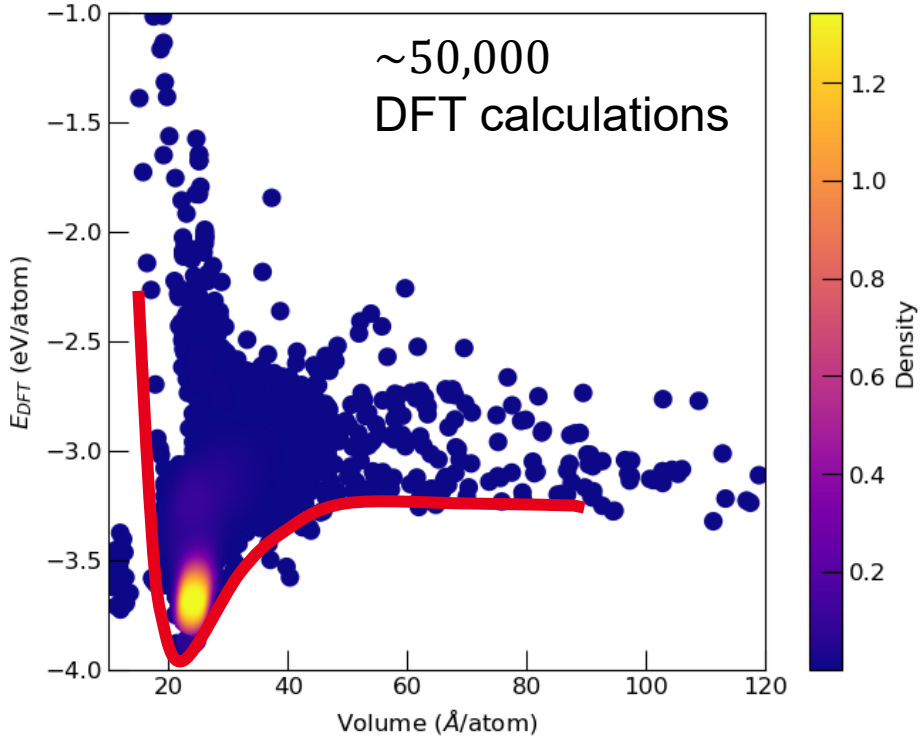
https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.hist.html



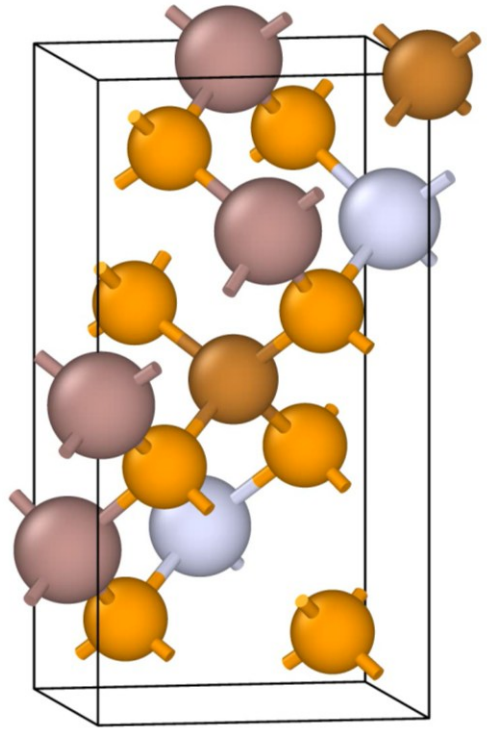
Create the appropriate data set

How a data set looks like?

- Create a data set by performing DFT calculations to investigate the material properties of $AgCuGaSe_2$ thin film solar cells



Volume per atom versus the energy per atom for the data set of $AgCuGaSe_2$ structures



The 1% of the total structures

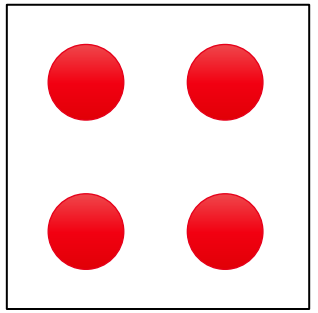
How to construct a database? (supervised learning)

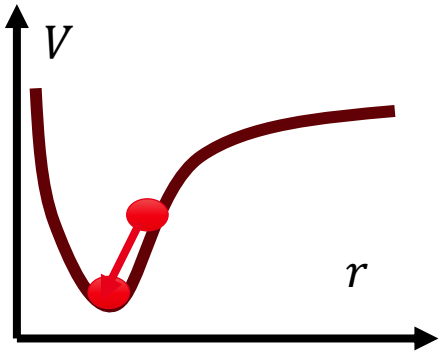
TECHNISCHE UNIVERSITÄT DARMSTADT

Crystalline structures:

→Taken from a crystal structure database (e.g. ICSD or MaterialsProject)

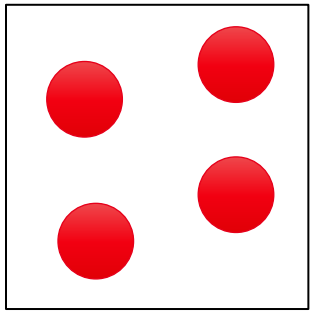
Ground state



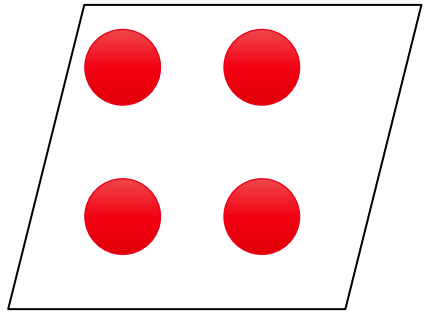


Generate several structures by deforming of the cell and displacing atoms

Displaced atoms



Deformed cell



This induces forces in the structures, which are helpful for the potential to learn the behaviour of the potential

How to construct a database? (Unsupervised Learning)

TECHNISCHE UNIVERSITÄT DARMSTADT

Amorphous and liquid structures

→Generate structures with

1. Classical interatomic potentials
2. DFT
3. A first version of your ML potentials

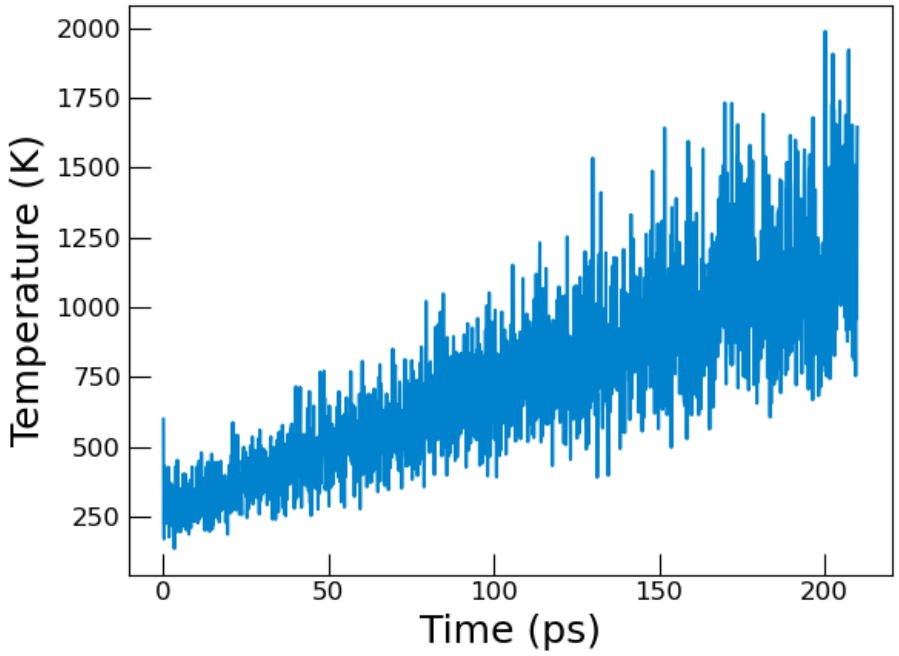
in molecular dynamics simulations.

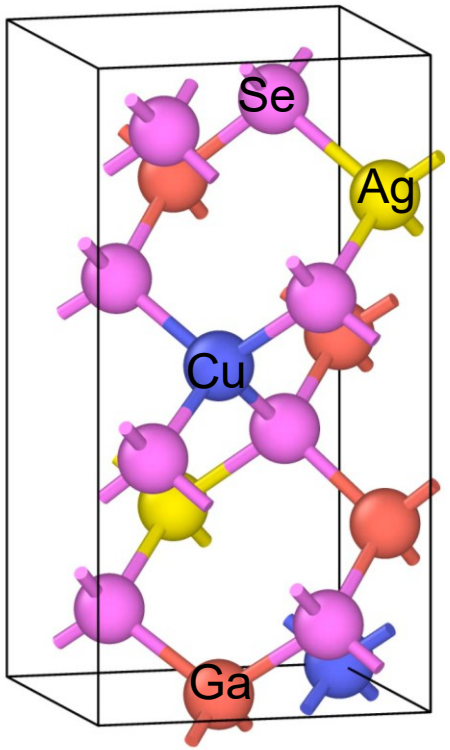
General protocol:

1. Take some crystalline structure and heat it to high temperatures (melting)
2. Quench the melt down to ambient temperatures (amorphous)

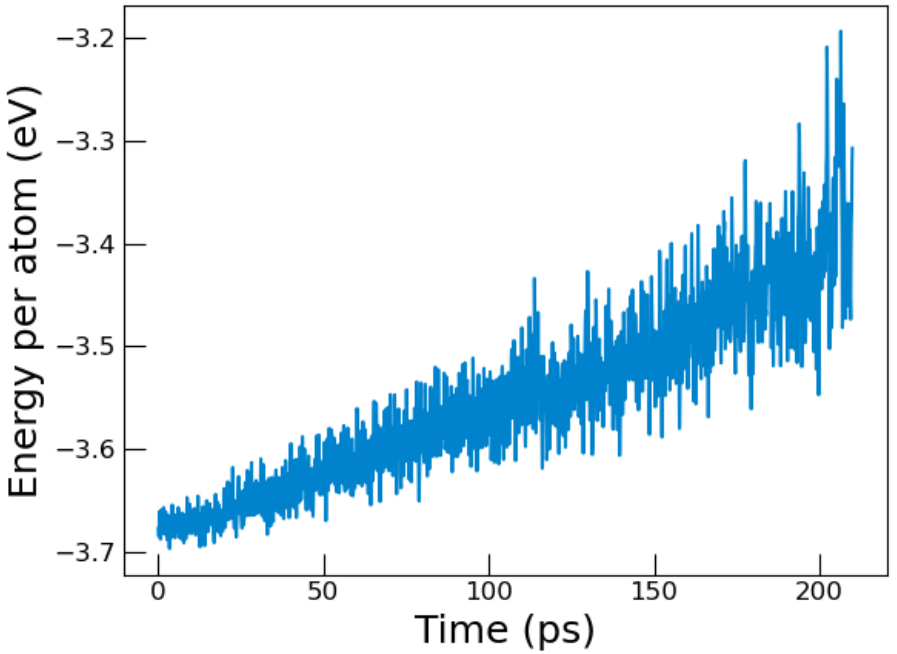
Molecular Dynamics (Unsupervised Learning)

TECHNISCHE UNIVERSITÄT DARMSTADT





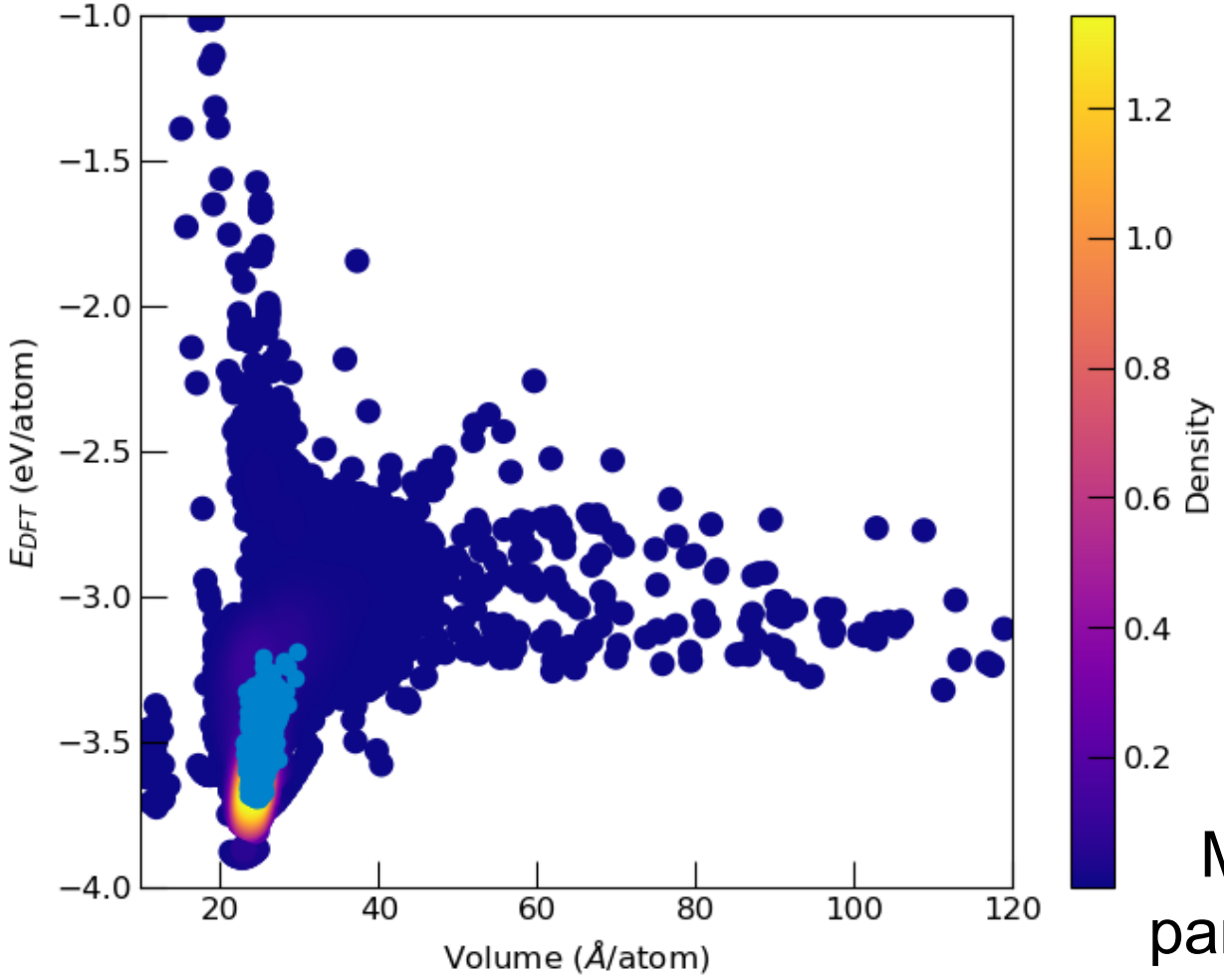
$Ag_2Cu_2Ga_4Se_8$



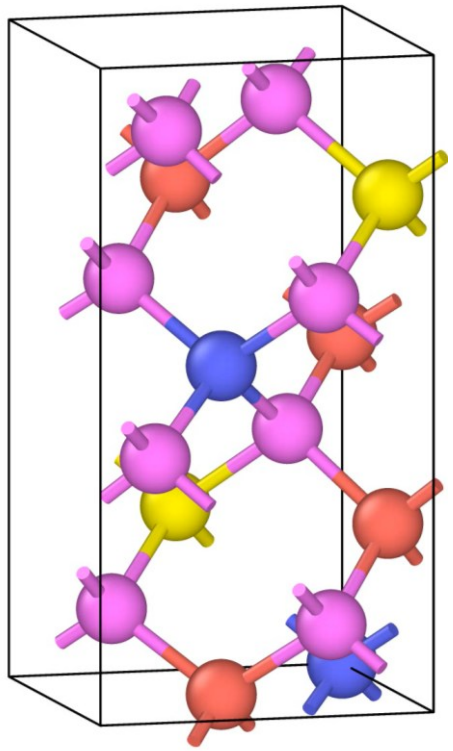
Consider the NPT MD simulation with increasing temperature – start from 300K and increase up to 1200K

Molecular Dynamics (Unsupervised Learning)

TECHNISCHE UNIVERSITÄT DARMSTADT



$Ag_2Cu_2Ga_4Se_8$



MD simulations help us to explore the parameter space to populate our data set

How to construct a database? (Active learning)

TECHNISCHE UNIVERSITÄT DARMSTADT

Uncertainty indicator:

Automatic recognition of structures, which are not known to the potential

→Impose a query to select these structure

→Calculate energy and forces by DFT and feed them to the database

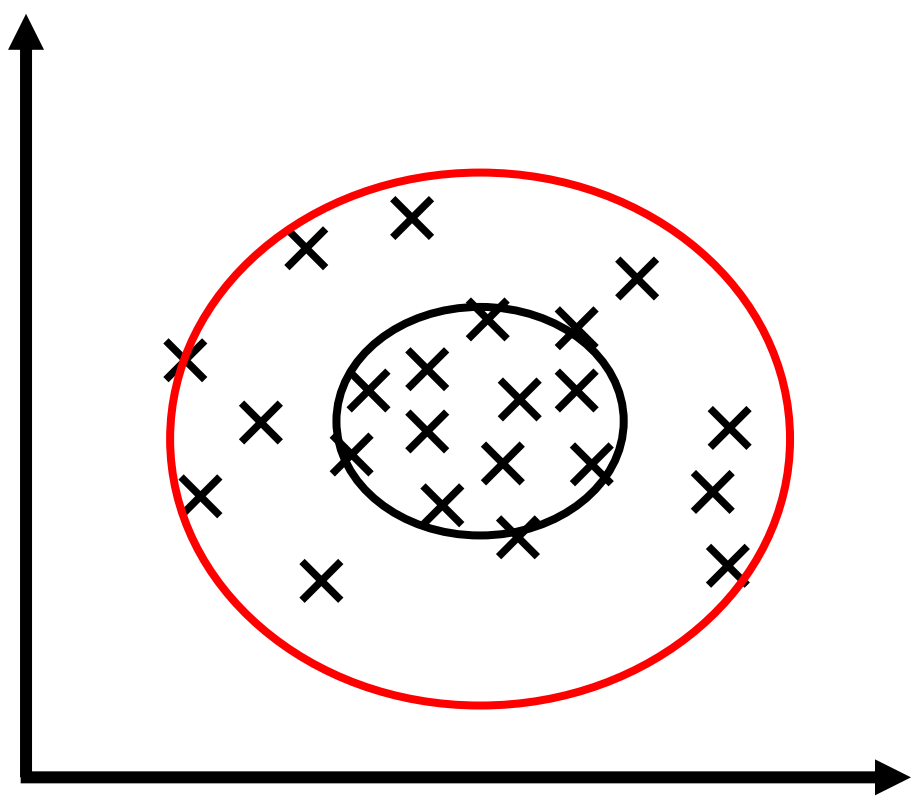
→Train the ML potential again

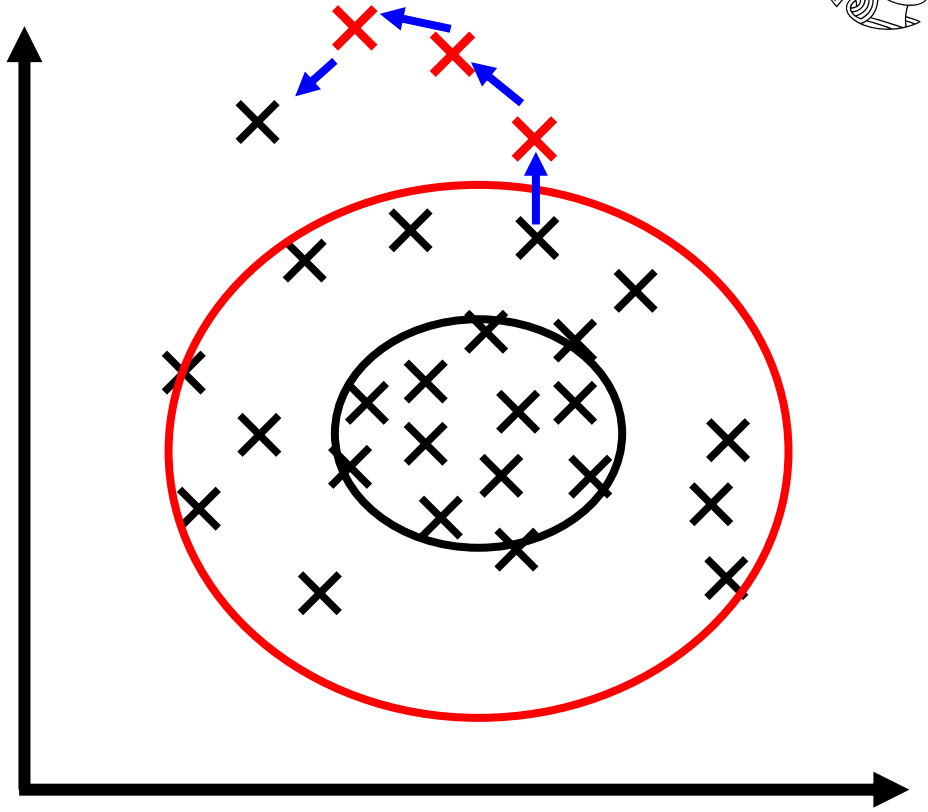
….Repeat until you can describe the desired property

Various types of uncertainty indicators implemented.

Active Learning – Extrapolation parameter

TECHNISCHE UNIVERSITÄT DARMSTADT

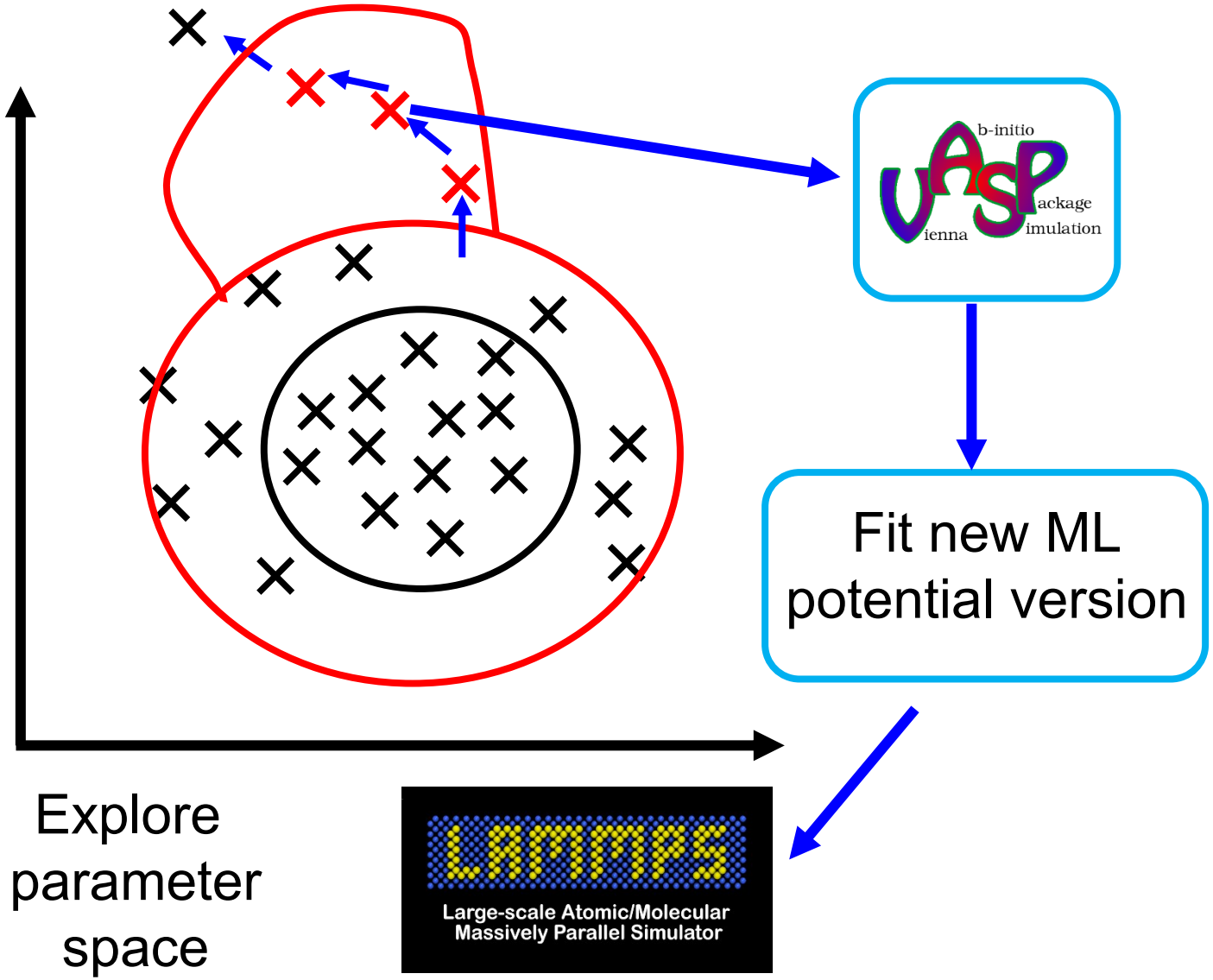




Black circle: Volume= V_0
Red circle: Volume= $5V_0$

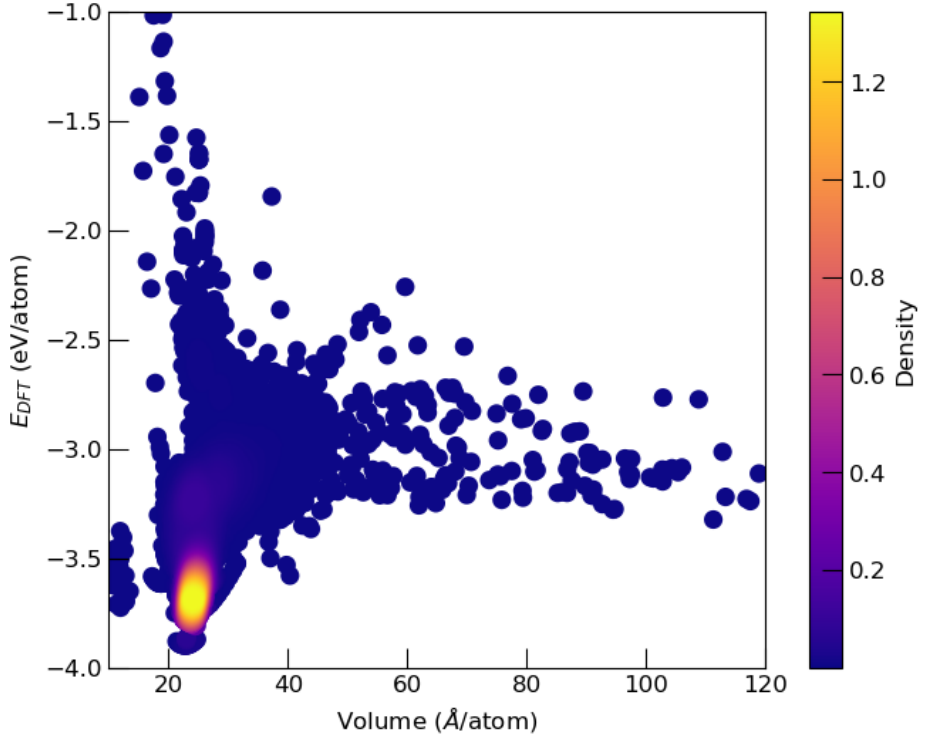
If Volume $> 25V_0$, then the MD simulation stops

Active Learning

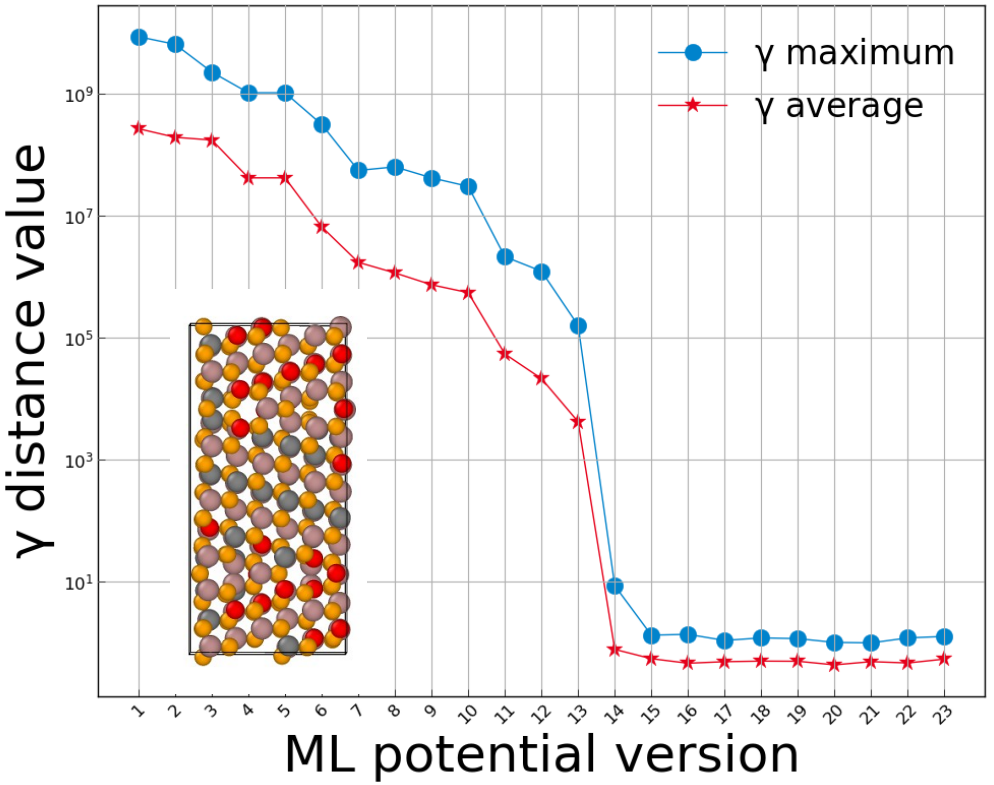


"Iteratively" repeat until we get a desirable ML potential

ML potential versions during active learning



Volume Vs Energy plot for the $AgCuGa_2Se_4$ during active learning

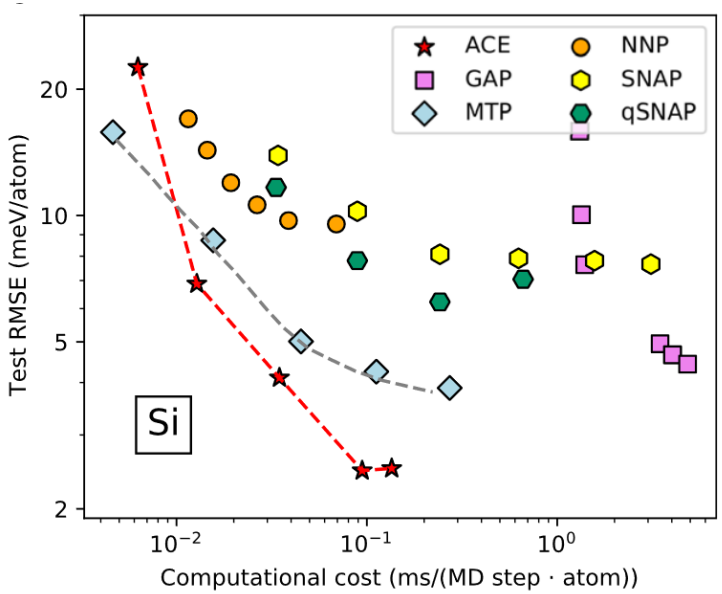


Uncertainty indicator value to assess the structure in question during active learning rounds

Recent approaches for ML potentials

TECHNISCHE UNIVERSITÄT DARMSTADT

- Gaussian approximation potentials and Behler-Parrinello neural network potentials have been developed ~10 years ago
- By now there are more “advanced” approaches
- One of these approaches is the atomic cluster expansion



Drautz, PRB 99, 014104 (2019)
Lysogorskiy, npj CM 7, 97 (2021)

Atomic cluster expansion

- The atomic cluster expansion is a complete formalism, contrary to SOAP and ACSF

- An arbitrary atomic property can be expressed by a sum over basis functions and their corresponding fitting prefactors

$$\varphi_i(\sigma) = \sum_{K\boldsymbol{nl}} \boldsymbol{c}_{\boldsymbol{nl}}^{(K)} \boldsymbol{B}_{i\boldsymbol{nl}}^{(K)}$$

Cluster with K bonds

n and l are parameters for the basis functions

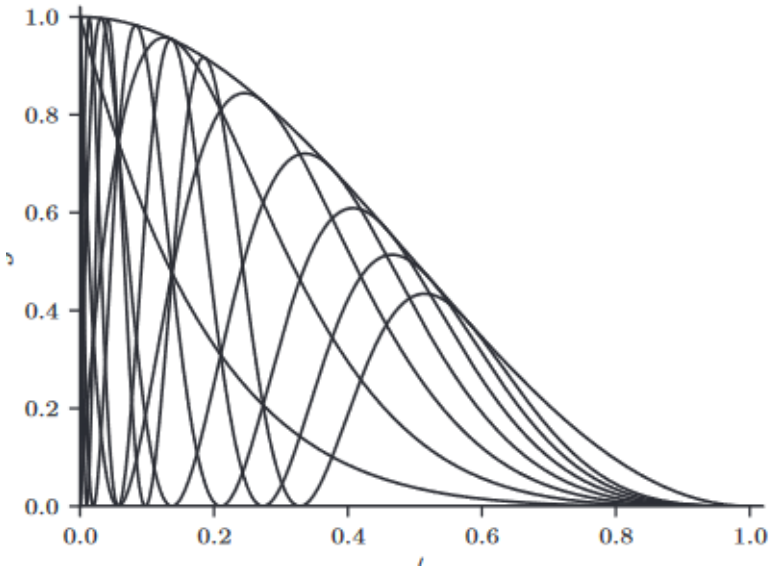
- The basis functions can be divided into 2-body, 3-body, …, n-body functions

Drautz, PRB 99, 014104 (2019)

Atomic cluster expansion

- Each single bond between two atoms is expressed by a product of a radial basis (Chebyshev polynomials or Bessel functions) and an angular basis (Spherical harmonics)

Radial basis (Chebyshev polynomials)



$$\varphi_{(nlm)}(r) = \sqrt{4\pi} R_{nl}(r) Y_l^m(\vec{r})$$

The basis functions are products of the atomic base:

$$A_{i(nlm)} = \sum_j \varphi_{(nlm)}(\overrightarrow{r_{ji}})$$

Drautz, PRB 99, 014104 (2019)

Code example - Tutorial

TECHNISCHE UNIVERSITÄT DARMSTADT

- We will use the code pacemaker to fit a simple ACE potential
- We use a subset of a database for silica for this (various quartz structures) taken from npj Computational Materials 8, 90 (2022)
- We will run a simple MD in ASE with this potential
- Pacemaker documentation can be found here: https://pacemaker.readthedocs.io

Pacemaker input file

```
cutoff: 5.0 # cutoff for neighbour list construction
seed: 42  # random seed

########################################################################
## Metadata section
########################################################################
metadata:
  origin: "Automatically generated input"

########################################################################
## Potential definition section
########################################################################
potential:
  deltaSplineBins: 0.001
  elements: ['O', 'Si']

  embeddings:
    ALL: {
      npot: 'FinnisSinclairShiftedScaled',
      fs_parameters: [ 1, 1, 1, 0.5 ],
      ndensity: 2,
    }

  bonds:
    ALL: {
      radbase: SBessel,
      radparameters: [ 5.25 ],
      rcut: 5.0,
      dcut: 0.01,
      NameOfCutoffFunction: cos,
    }

  functions:
    number_of_functions_per_element: 200
    UNARY:   { nradmax_by_orders: [ 15, 6, 4, 3, 2, 2 ], lmax_by_orders: [ 0 , 3, 3, 2, 2, 1 ]}
    BINARY:  { nradmax_by_orders: [ 15, 6, 3, 2, 2, 1 ], lmax_by_orders: [ 0 , 3, 2, 1, 1, 0 ]}
    TERNARY: { nradmax_by_orders: [ 15, 3, 3, 2, 1 ],    lmax_by_orders: [ 0 , 2, 2, 1, 1 ], }
    ALL:     { nradmax_by_orders: [ 15, 3, 2, 1, 1 ],    lmax_by_orders: [ 0 , 2, 2, 1, 1 ] }

```

Maximum cutoff of the potential
Random Seed for split of train and test set

Parameter for tabulated functions (do not change)
Elements, which are part of the database

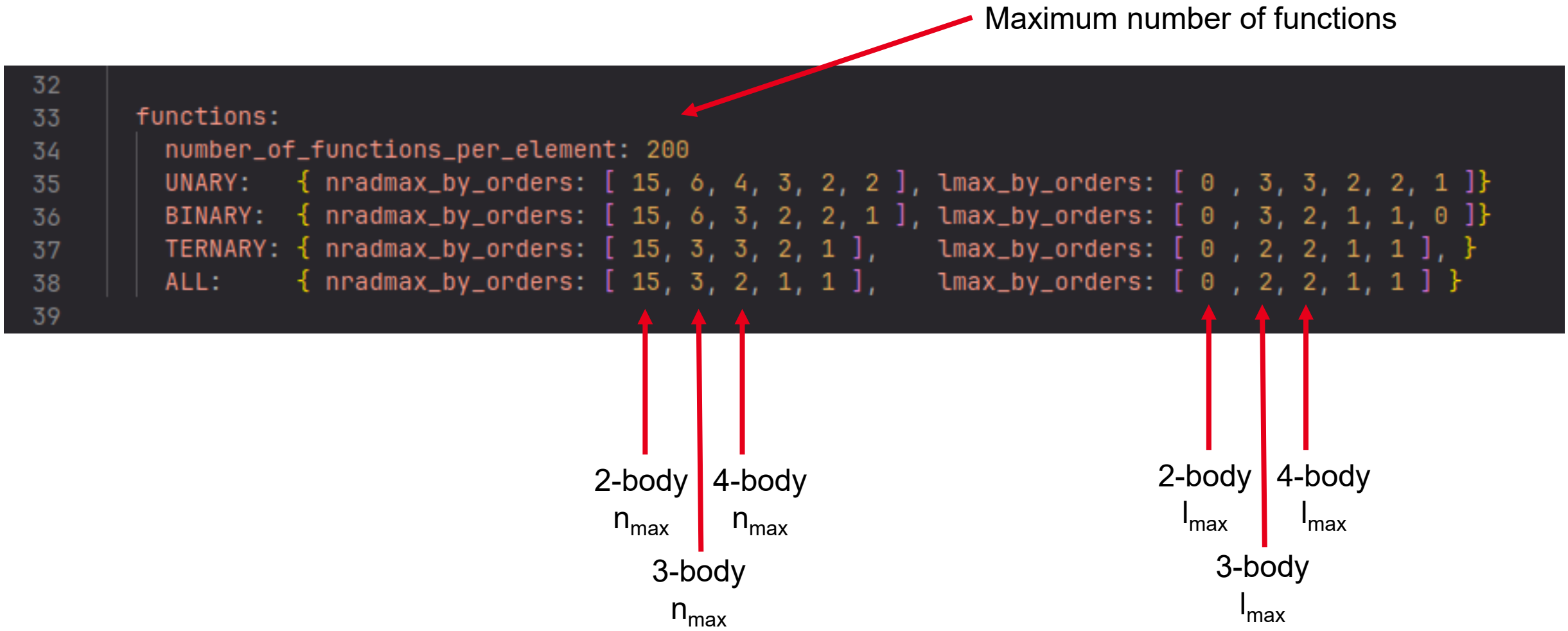
Define embeddings (non-linearity of the potential)

fs_parameters: $1 \cdot \left(\varphi_i^{(1)}\right)^1 + 1 \cdot \left(\varphi_i^{(2)}\right)^{0.5}$

Define radial basis (Bessel functions)
Define cutoff for specific bonds (normally equal to the maximum cutoff)
Define cutoff functions and other parameters (do not change)

Define basis functions

Pacemaker input file



Pacemaker input file

```yaml
##################################################################
## Dataset specification section
##################################################################
data:
  filename: data/training_SiO2.pckl.gzip       # force to read reference pickled dataframe from given file
  test_size: 0.1


##################################################################
## Fit specification section
##################################################################
fit:
  loss: { kappa: 0.3, L1_coeffs: 1e-8,  L2_coeffs: 1e-8}
  # if kappa: auto, then it will be determined from the variation of energy per atom and forces norms in train set



  optimizer: BFGS # or L-BFGS-B

  ## maximum number of minimize iterations
  maxiter: 300

  ## additional options for scipy.minimize
  #  options: {maxcor: 100}

  ## Automatically find the smallest interatomic distance in dataset  and set inner cutoff based on this
  #  repulsion: auto

  #  ladder_step: 100
  #  ladder_type: power_order

##################################################################
## Backend specification section
##################################################################
backend:
  evaluator: tensorpot
  batch_size: 100
  display_step: 50

```

Path to the database
Test set size (randomly selected)

Loss function (weights) and regularization settings

Optimizer (BFGS recommended) and number of iterations

Batch size, of memory issue are encountered this needs to be decreased

Recipe for a good ML potential

TECHNISCHE UNIVERSITÄT DARMSTADT

DFT data

- Properly converged (cut off energy, wavenumber etc)
- Pick the correct correlation functional

Active learning

- Patience! Might run to a local minima – rounds with few new structures

Fitting the dataset to ML algorithm

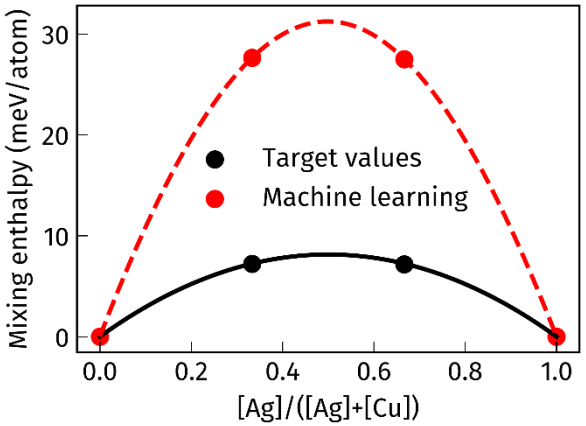
- Pay attention to kappa parameter – 500 steps kappa to 0.95
  500 steps kappa to 0.05
- Do not overfit
- Cut-off radius

Be careful!!!

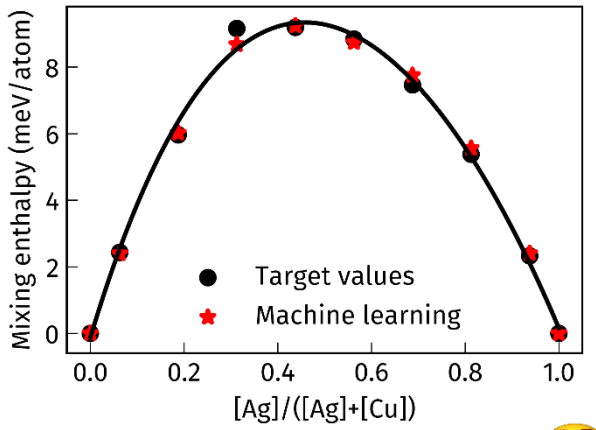
TECHNISCHE UNIVERSITÄT DARMSTADT

1) Dataset quality

The model predictions are as good as the dataset quality



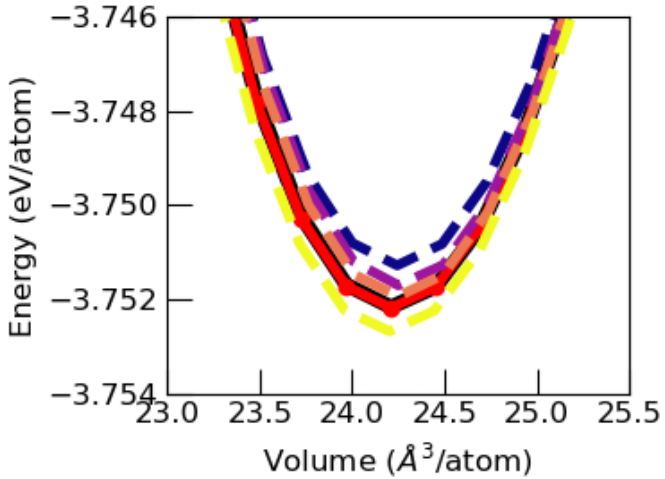
Poor quality DFT dataset



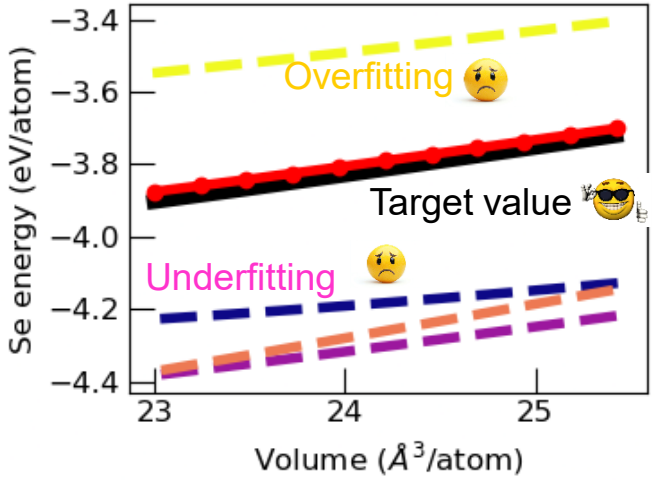
High quality DFT dataset

2) Fitting process

The outcome of the fitting is verified by physical quantities



Energy - volume curve, all ML models have similar outcome



Feedback

TECHNISCHE UNIVERSITÄT DARMSTADT

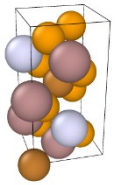
TECHNISCHE UNIVERSITÄT DARMSTADT

Validate and utilize ML Potentials

Machine learning approach

TECHNISCHE UNIVERSITÄT DARMSTADT

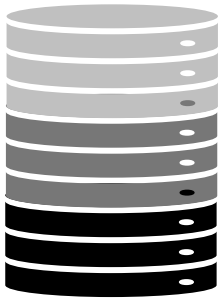
Data set



$\hat{H}\psi = E\psi$

Energy and forces

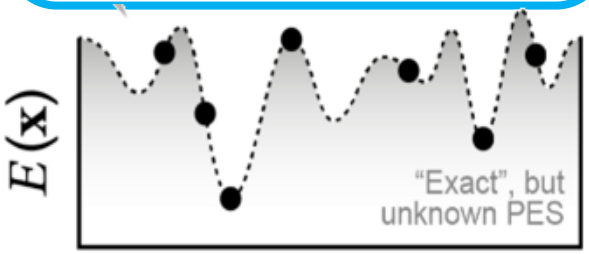
→ Atomic cluster expansion ("descriptors") → Machine Learning Potential → Study material properties



50,000 structures

$10 - 100$ atoms

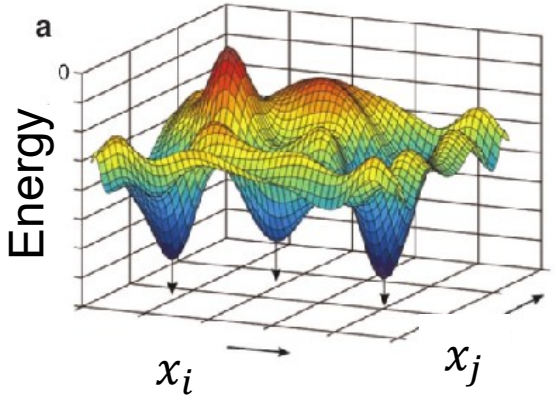
4×10^6 CPU hours



X (3N dimensional)

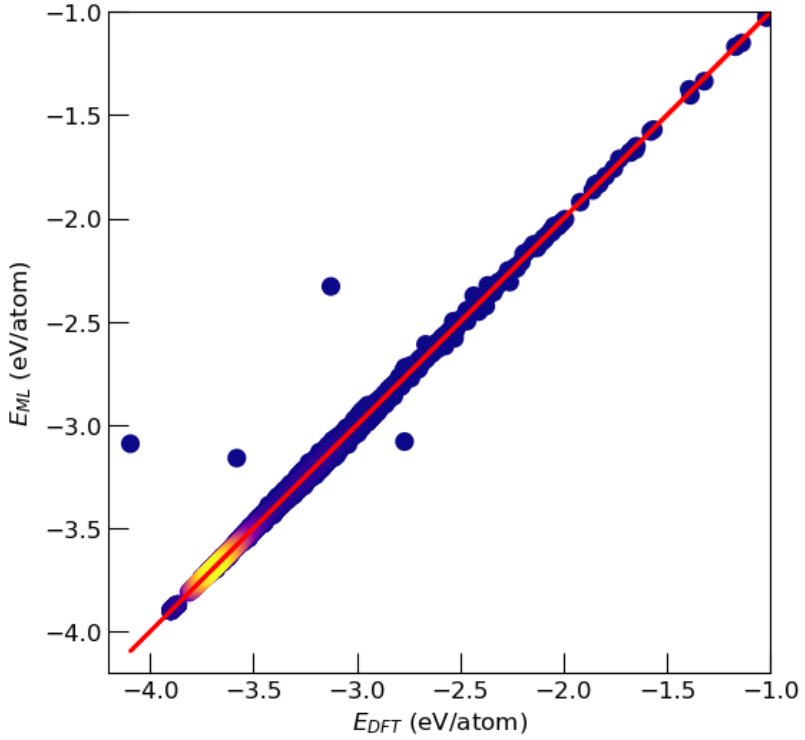
$$\begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} \rightarrow \begin{pmatrix} q_1 \\ q_2 \\ \vdots \\ q_n \end{pmatrix}$$

Energy landscape

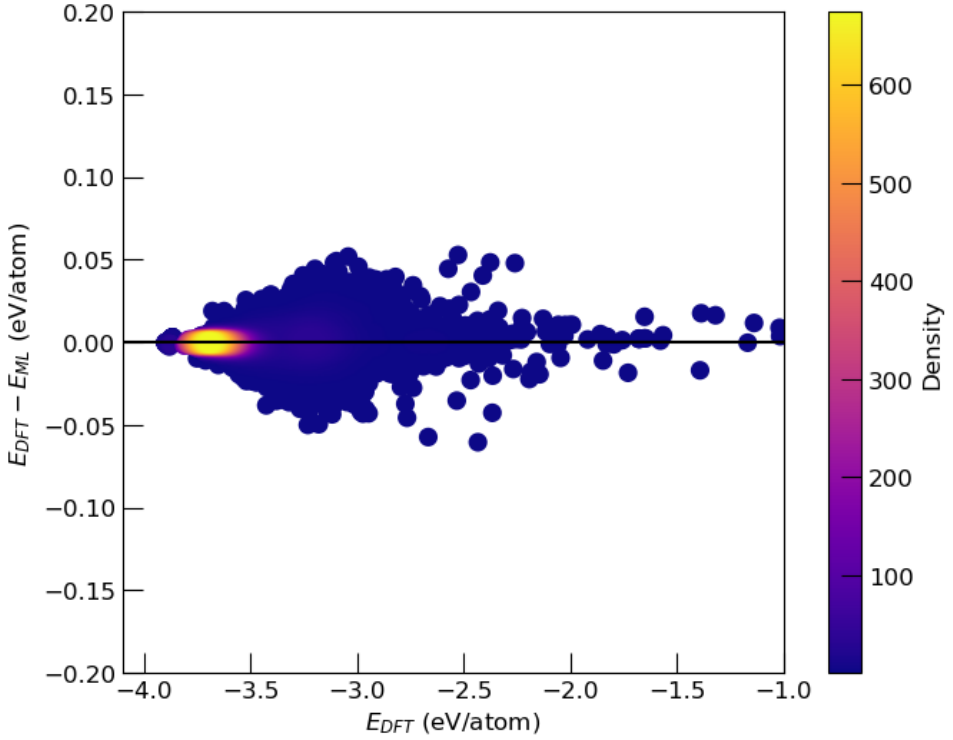


R. Drautz, Phys. Rev. B 99, 014104 (2019)

Validating a Machine Learning Potential



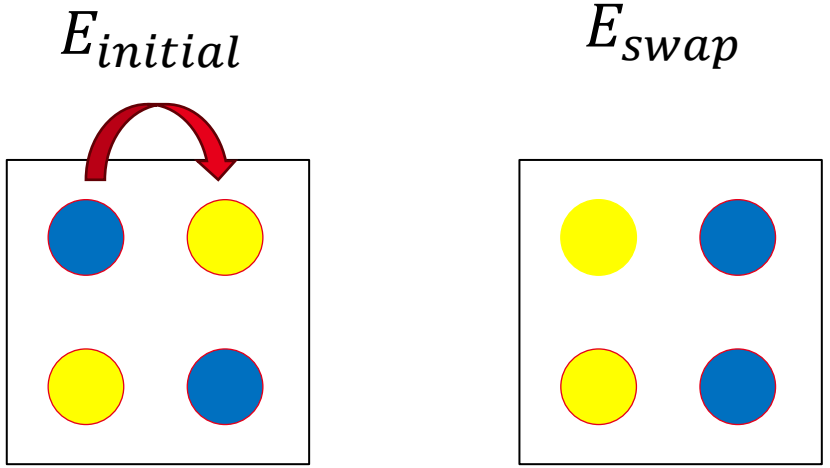
Energy per atom comparing DFT with ML results



ML energy error for the given DFT calculation result

MD and Monte Carlo calculations

- Monte Carlo method is used to explore the parameter space
- Randomly selects a Ag and Cu pair
- Computes the energy change ΔE after the swap
- Uses Metropolis algorithm to accept or reject the swap:
  - If $\Delta E < 0 \rightarrow$ Accepts (lowers energy)
  - If $\Delta E > 0 \rightarrow$ Accepts with probability $P = \exp(-\Delta E / k_B T)$



Initial

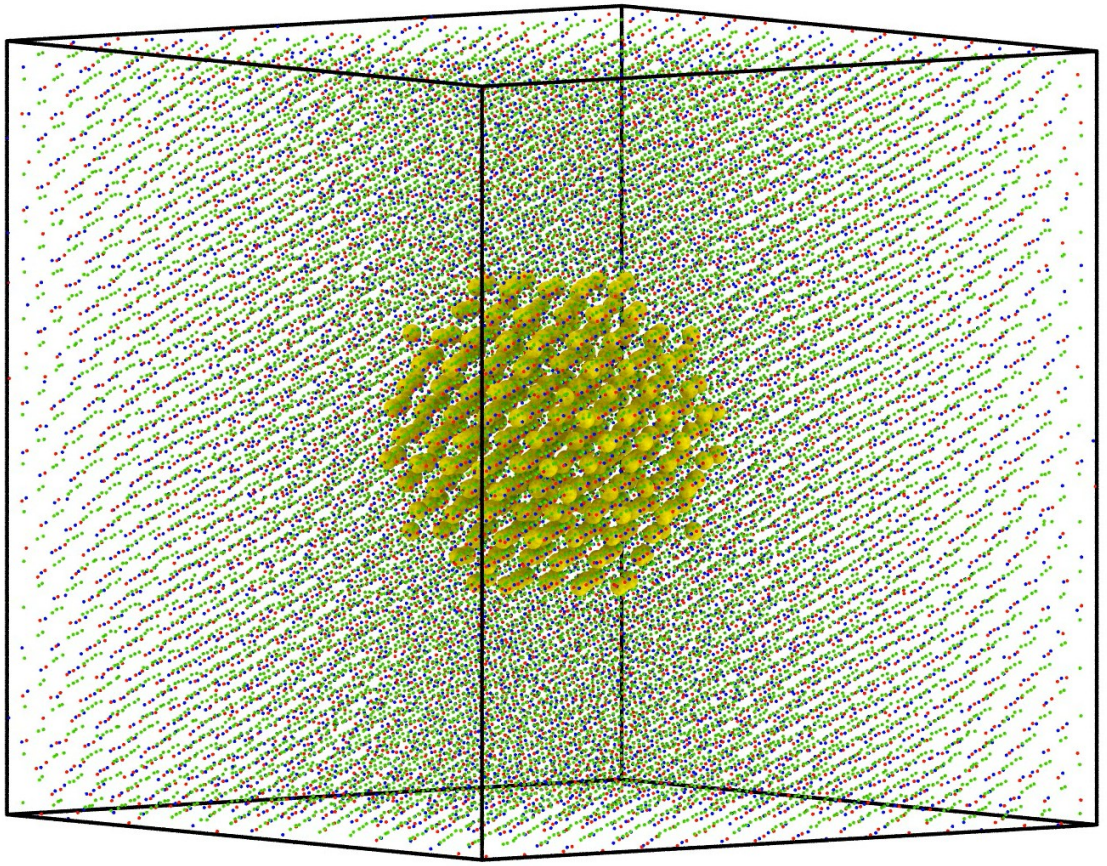
Attempted swap

$$\Delta E = E_{swap} - E_{initial}$$

ML potential to investigate material properties

TECHNISCHE UNIVERSITÄT DARMSTADT

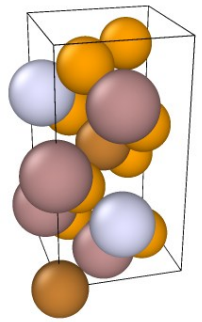
Spherical inclusion of $AgGaSe_2$ in $CuGaSe_2$ matrix, MD+MC simulation at room temperatures. The Ag atoms are presented for better visibility



Workflow to develop the ML potential

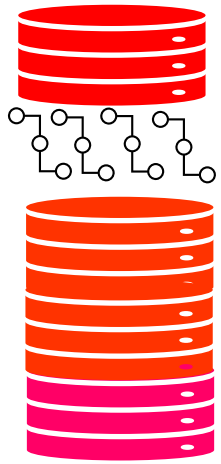
TECHNISCHE UNIVERSITÄT DARMSTADT

Create an initial data set

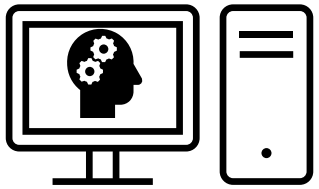


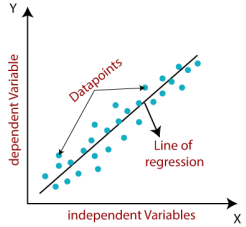
$AgCuGa_2Se_4$

Training data set



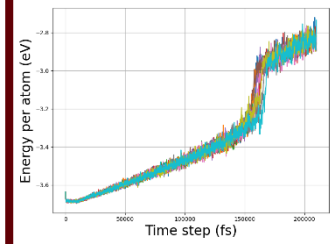
Machine Learning





Molecular Dynamics

LAMMPS
Large-scale Atomic/Molecular Massively Parallel Simulator



Ab initio calculations

b-initio VASP ackage ienna imulation

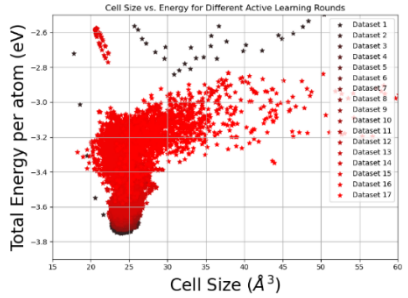
Update dataset



Repeat

Interatomic Potential

Final data set



MLIP validation plot

